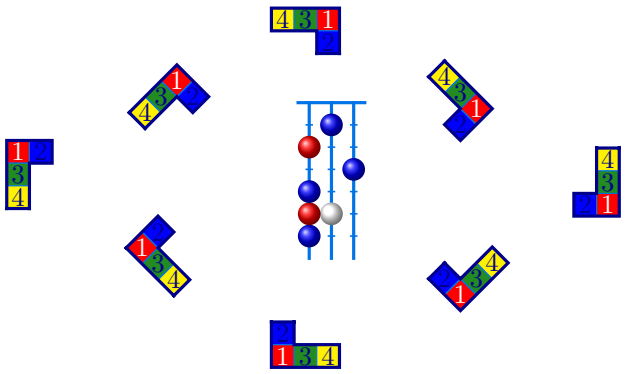
aTableau

Andrew Mathas

Version 2.0.alpha

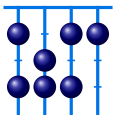
Contents



1. Introduction

The package provides commands for drawing common objects in algebraic combinatorics and representation theory, together with styling options and a key-value interface for additional customization. Under the hood, everything is drawn using `TikZ` (and LaTeX3). These commands can be used either as standalone commands or as part of a `tikzpicture` environment. This package grew out of my own research, so it does all of the things that I want it to do. On the other hand, it is written as flexibly as possible in the hope that it will draw the diagrams that others need.

For the impatient, here is a basic example that shows how to use the main commands provided by the aTableau package:



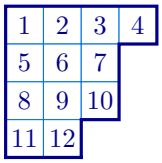
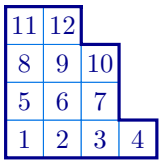
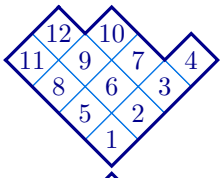
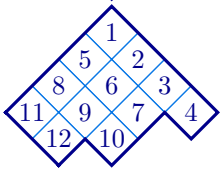
1	2	3	9
4	5	6	10
7	8		

```
\Abacus{4}{4^3,2,1^2,0}
\Tableau{1239,456{10},78}
```

If you already know what tableaux and abacuses are, then you can probably work out the (basic) syntax of these two commands from this example and start using the package. If you do not know what tableaux and abacuses are, then you probably don't need this package! In any case, you will not find the precise mathematical definitions of these objects in this manual, so please consult textbooks like [**?**, **?**] if you need them. For more advanced usage, such as adding styling, I recommend skimming through the manual, and looking at the many examples. Chapter 4 gives quick summary of all aTableau commands and their options.

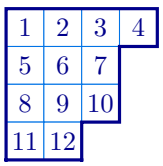
The aTableau commands are intended to be easy to use and easy to customise. For example, partitions can be entered either as a (weakly decreasing list of non-negative) integers, or using exponential notation, or as a sequence of beta numbers (for the `\Abacus` command), and tableaux are entered as comma-separated words. The commands support the main conventions for drawing tableaux and diagrams.

By way of example, the following table shows how to draw a Young diagram of shape $\lambda = (4, 3^2, 2)$ using the four tableaux conventions supported by aTableau:

Convention	aTableau command	Diagram
English	`\Tableau[english]{1234,567,89{10},{11}{12}}` (the default)	
French	`\Tableau[french]{1234,567,89{10},{11}{12}}`	
Ukrainian[1]	`\Tableau[ukrainian]{1234,567,89{10},{11}{12}}`	
Australian[2]	`\Tableau[australian]{1234,567,89{10},{11}{12}}`	

This table shows how the commands provided by aTableau work: there are basic commands for drawing diagrams, and these diagrams can be embellished using optional arguments, which use a key-value interface.

The different conventions for diagrams, and tableaux, are listed in order of (my perception of) their popularity in the literature. By default, the *English* convention is used, so we can omit *english*:



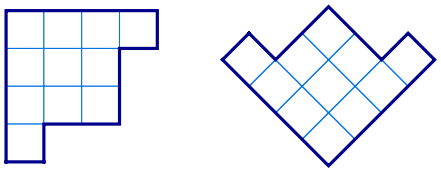
1	2	3	4
5	6	7	
8	9	10	
11	12		

```
\Tableau{1234,567,89{10},{11}{12}}
```

for the first tableau in the table. These four conventions are used in exactly the same way with the `\Diagram` command:

[1] Some authors call this the Russian convention.

[2] I have not seen a name for this convention in the literature. Calling this the *Australian convention* seems apt.



```
\Diagram{4,3,3,1}
\Diagram[ukrainian]{4,3^2,1}
```

This example shows that partitions can either be typed out in full, as a comma-separated list of positive integers, or using exponential notation, which is usually shorter and easier to read.

To use the package, add the following line to your document preamble:

```
\usepackage[options]{atableau}
```

Here, the *options* is an optional comma-separated list of aTableau options, or keys — we describe the possible options below. For example, to make the tableaux and Young diagrams commands use the French convention, by default, you could write:

```
\usepackage[french]{atableau}
```

You can change the default options at any point in your document using the `\aTabset` command. Options can also be applied to individual commands.

Each of the aTableau commands is designed to be easy to use and easy to read, and it is highly customisable. Each command accepts an optional list of key-value options that modify the diagrams. In addition, optional styling can be applied to entries of a tableau and the beads in an abacus. Each command can be used as a standalone object in a document, or they can be incorporated into a larger `tikzpicture` environment, making makes it possible to use these commands as part of more complicated diagrams.

The general syntax of an aTableau command is:

```
\Command (x,y) [options] {...specifications...}
```

The (x, y)-Cartesian coordinates are optional, being necessary if (and only if), the diagram is part of a `tikzpicture` environment. Similarly, the key-value *options* control the style and appearance of the final diagram.

The possible options, or keys, for the aTableau commands are described in detail below, with examples. This section emphasises only some of these options together with the fact that every key can be set "gobally" (inside the current LaTeX group), by using `\aTabset{...options...}`. As discussed already, the default options for the document can also be set via `\usepackage[options]{atableau}`.

Many of the aTableau options are specific to the particular diagrams being drawn. The full list of options is given in Chapter 4. The following options are common to all aTableau commands.

align :
Sets the baseline for vertically aligning diagrams. The possible options are `align=top`, `align=center`, and `align=bottom`, which aligns the baseline of the diagram with the top, centre or bottom of the surrounding objects, respectively. By default, aTableau are centered.

name :
Set the prefix for the TikZ *named anchors* for the boxes and beads in aTableau diagrams.

scale, xscale, yscale :
Use scale to rescale all aTableau diagrams. Use xscale to rescale in the x-direction and yscale to rescale in the y-direction.

tikzpicture :
When used without Cartesian coordinates, all aTableau commands are implicitly drawn inside a `tikzpicture` environment. Use tikzpicture=keys to set the optional argument of this environment:

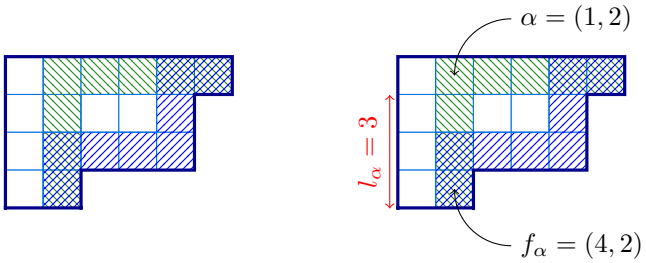
`\begin{tikzpicture}[ keys ]...\end{tikzpicture}`.

tikz before, tikz after :
Use the tikz before and tikz after keys to inject TikZ code into the `tikzpicture` environment, before and after the diagram drawn by aTableau.

With boolean options, which are set to true or false, the value can typically be omitted. For example, border is the same as border=true (and no border is the same as border=false). In all of the options, American spelling variations, such as *center* (instead of *centre*), *color* (instead of *colour*) et cetera, are tolerated. All of the colours in aTableau diagrams can be changed using the key-value options. The names of the underlying base colours used by aTableau are given in Section 4A.

The next example shows how, how to use the `\Diagram` command inside a `tikzpicture` environment by supplying Cartesian coordinates.

1.4
```
% \usetikzlibrary{patterns}
\tikzset{
  B/.style={pattern=north east lines, pattern color=blue},
  G/.style={pattern=north west lines, pattern color=ForestGreen},
}
\Diagram[ribbons={(G)16ccccrrr, (B)16crrcccr}]{6,5^2,2} % unlabelled diagram
\qquad\qquad
\begin{tikzpicture} % with labels using a tikzpicture environment
  \Diagram(0,0)[ribbons={(G)16ccccrrr, (B)16crrcccr}]{6,5^2,2}
  % add the labels
  \draw[->](1.5, 0.5)node[right]{$\alpha=(1,2)$} to [out=180,in=90]  (B-1-2.base);
  \draw[->](1.5,-2.5)node[right]{$f_\alpha=(4,2)$} to [out=180,in=270] (B-4-2.base);
  \draw[red,<->]([xshift=-1mm]B-1-1.south west)
     --node[rotate=90,above]{$l_\alpha=3$}([xshift=-1mm]B-4-1.south west);
\end{tikzpicture}
```

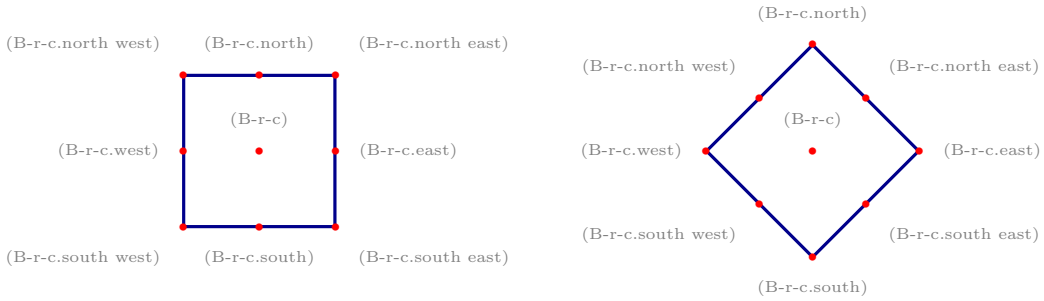


The diagram is drawn twice, both outside and inside a `tikzpicture` environment. The first diagram, which does not have labels, emphasizes that most of the right-hand diagram is drawn by the `\Diagram` command, even though most of the commands in the example are TikZ commands. In particular, the shading of the boxes in the $(1,2)$-*hook* and $(1,2)$-*rim hook* is done by the ribbons key. (In fact, this diagram could be drawn without using a `tikzpicture` environment by adding the `\draw` -commands to the tikz after key.)

The top of the last example defines two TikZ-styles, B and G, which add the blue and green coloured hatchings of the two hooks, respectively. Inside the `tikzpicture` environment, the Young diagram is placed at position $(0,0)$ by the `\Diagram` command. The three `\draw` commands are used to add the three labelled arrows to the picture. Please consult the very readable TikZ manual for more details on the TikZ-commands in this example.

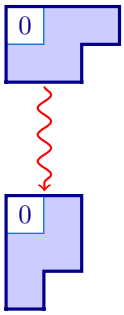
From the viewpoint of aTableau, the most important point of the last example is that the first `\Diagram` command is a standalone diagram, whereas the second one is part of a `tikzpicture` environment because it is equipped with the Cartesian coordinate $(0,0)$. The second important point is that this example uses four *named coordinates*: (B-1-1), (B-1-2), (B-4-1), and (B-4-2). When aTableau draws a tableau, or a diagram, it creates a TikZ named coordinate (B-r-c) for every box in the diagram, where (B-r-c) is the box in row r and column c. These named coordinates, or anchors, are *only* defined for the boxes in the tableau. Giving finer control, the following standard TikZ *anchors* are available:

(1.1)



You can change the B in the name of these anchors using the name key. Similar anchors are created for abacus beads. These *anchors* are a standard part of TikZ. If you change the shape of the node, then the available anchors may change; for more information see the TikZ manual.

In the example above, the Cartesian coordinate $(0, 0)$ is necessary because the \Diagram command is used inside a tikzpicture environment, but the choice of coordinate is not important in the sense that changing the coordinate will not change the picture. Cartesian coordinates become more meaningful when the tikzpicture environment has more components. In the next example, notice the use of name=A to make the named coordinates for the first diagram take the form (A-r-c), instead of the default node names (B-r-c), which are used for the second diagram.

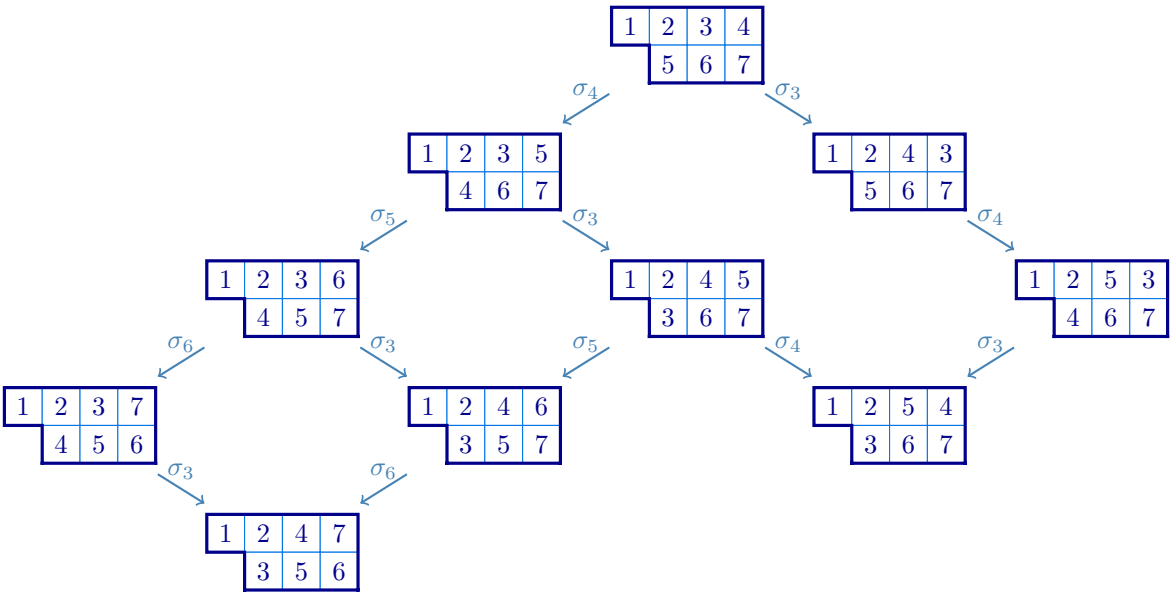


1.5
```
% \usetikzlibrary{decorations.pathmorphing}
\begin{tikzpicture}[wave/.style={thick,red,->,decorate,decoration=snake}]
  \aTabset{ribbon style={fill=blue!20}}
  \Diagram(0,0)[e=2,entries=residues,ribbons={13crc},name=A]{3,2}
  \Diagram(0,-2.5)[e=2,entries=residues,ribbons={12rcr}]{2^2,1}
  \draw[wave]([shift={(0,-0.05)}]A-2-1.south east)
              --([shift={(0,0.05)}]B-1-1.north east);
\end{tikzpicture}
```

The final example in the introduction uses \ShiftedTableau inside a *matrix of nodes*:

1.6
```
% \usetikzlibrary{matrix}
\begin{tikzpicture}[scale=0.8, arr/.style={->,SteelBlue, thick}]
  \matrix (M)[matrix of nodes,row sep=4mm,column sep=4mm]{
      &    &    & \ShiftedTableau{1234,567} \\
      &    & \ShiftedTableau{1235,467}
      &    & \ShiftedTableau{1243,567}\\
      & \ShiftedTableau{1236,457}
      &    & \ShiftedTableau{1245,367}
      &    & \ShiftedTableau{1253,467}\\
    \ShiftedTableau{1237,456}
      &    & \ShiftedTableau{1246,357}
      &    & \ShiftedTableau{1254,367}\\
      & \ShiftedTableau{1247,356}\\
   };
   \draw[arr](M-1-4)--node[above]{$\sigma_4$}(M-2-3);
   \draw[arr](M-1-4)--node[above]{$\sigma_3$}(M-2-5);
   \draw[arr](M-2-3)--node[above]{$\sigma_5$}(M-3-2);
   \draw[arr](M-2-3)--node[above]{$\sigma_3$}(M-3-4);
   \draw[arr](M-2-5)--node[above]{$\sigma_4$}(M-3-6);
   \draw[arr](M-3-2)--node[above]{$\sigma_6$}(M-4-1);
   \draw[arr](M-3-2)--node[above]{$\sigma_3$}(M-4-3);
   \draw[arr](M-3-4)--node[above]{$\sigma_5$}(M-4-3);
   \draw[arr](M-3-4)--node[above]{$\sigma_4$}(M-4-5);
   \draw[arr](M-3-6)--node[above]{$\sigma_3$}(M-4-5);
   \draw[arr](M-4-1)--node[above]{$\sigma_3$}(M-5-2);
```

```
    \draw[arr](M-4-3)--node[above]{$\sigma_6$}(M-5-2);
\end{tikzpicture}
```



Notice that even though the `\SkewTableau` commands are being used inside a `tikzpicture` environment they do not need coordinates because the skew tableaux are not explicit components in the environment, being all (implicitly) inside TikZ-nodes.

The following sections describe the commands defined by the aTableau package and how to use them. We start with the `\Tableau` command because all of the diagram and tableau commands, except for `\RibbonTableau`, are derived from `\Tableau`.

2. Tableaux and Young diagrams

Partitions, which are weakly decreasing sequences of non-negative integers, are fundamental objects in algebraic combinatorics and representation theory. Rather than working with sequences of integers, it is often useful to visualise partitions as Young *diagrams*, which are arrays of *boxes*, or *nodes*, in the plane[3]. A *tableau* is a (Young) diagram where the boxes are labelled using some alphabet. Equivalently, a diagram is a tableau with empty labels.

For example, $\lambda = (4, 3, 3, 2, 0, 0, \dots)$ is a partition of 12, since the entries sum to 12. It is customary to omit the zeros when writing partitions and to use exponents for repeated parts, so we can write this partition in *exponential notation* as $\lambda = (4, 3^2, 2)$. Many authors identity a partition $\lambda = (\lambda_1, \lambda_2, \dots)$ with its *diagram*, which is the set $[\lambda] = \{\,(r, c) \,|\, 1 \le c \le \lambda_r \text{ for } r \ge 1\,\}$. When using the *English convention*, the diagram of λ is visualised as a left justified array of boxes in the plane, where the first row has 4 boxes, the next two lower rows have 3 boxes and the last row has 2 boxes. The *shape* of a diagram is the partition that gives the number of boxes in each row. Diagrams and tableaux drawn using the *French* convention are given by reflecting in a horizontal line above the diagrams, whereas the *Ukrainian* and *Australian* conventions are given by appropriately rotating these diagrams. All of these conventions are used in the literature, with some papers using more than one convention.

2A. **Tableaux.**

This section describes the `\Tableau` command, which draws tableaux or labelled diagrams. The syntax of this command is:

```
\Tableau (x,y) [options] {tableau entries}
```

where:

[3] In this manual, we normally refer to the entries of tableaux and diagrams as *boxes*, rather than *nodes*, to avoid any ambiguity with TikZ-nodes.

(x,y) :
The Cartesian coordinates (x,y) are needed if, and only if, the tableau is a component of a `tikzpicture` environment.

options :
Optional arguments. Described, with examples, below.

tableau entries :
The `tableau entries` are given as a comma-separated list, with commas separating rows and each *token*, or braced-group of tokens, being in a separate column. As we explain below, each tableau entry can be prefixed by (optional) TikZ-styling specifications.

We show by example how to use the `\Tableau` command, starting with basic usage and finishing with style. Inside `\Tableau`, the rows are separated by commas, with the columns given by the "letters" in the word for each row:

1	2	3	4
5	6		
7			
8			

α	γ	δ	q
x	y	η	
λ	μ		
π	$\sum$		

2A.1
```
\Tableau{ 1234, 56, 7, 8 }
\Tableau{ \alpha\gamma\delta q,
          xy\eta,
          \lambda\mu,
          \pi\sum }
```

The tableau specifications can be put on a single line, without any spaces. Alternatively, as shown here, spaces and line breaks can be added to improve readability. The one restriction is that you cannot have blank lines inside a `\Tableau` command. As this example suggests, by default, the entries in a tableau are typeset in mathematics-mode, but this can be changed using the text boxes key, which is described below.

As the examples above show, the tableau entries can be individual characters, or they can be TEX commands. Tableaux frequently contain entries that are not single characters, or given by commands. Such entries can be added to a tableau by enclosing them in braces:

1	3	10	11	$2x$
2	x^2			

2A.2
```
\Tableau{ 13{10}{11}{2x}, 2{x^2} }
```

There is one additional caveat for braced-entries: any *singleton* braced entry `{...}` in a row needs to be *double-braced*. Double-bracing is only necessary when you have a row that has only one column, and you have a multi-letter column entry. The need for double-bracing is a consequence of how LATEX3 detects braced commas in comma-separated lists.

The following example shows the difference between single and double-bracing for tableau entries.

1	3	4	5
8	10		
1	1		
1	2		

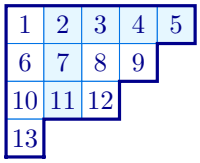
1	3	4	5
8	10		
11			
12			

2A.3
```
\Tableau{ 1345, 8{10}, {11}, {12} }
\Tableau{ 1345, 8{10}, {{11}}, {{12}} }
```

The 10's in these tableaux do not need to be doubled-braced because they appear in a row of length 2, whichi s greater than 1!. In contrast, because they are not doubled-braced, the 11 and 12 in the first tableau are both treated as being two characters in different columns in the first tableau. In the second tableau, the 11 and 12 each appear in a separate column because they are double-braced.

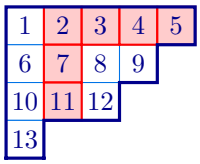
2A.1. *Adding style.* The aTableau package provides several different ways to add TikZ style specifications to the entries in tableaux, and to boxes in diagrams. This is made possible by adding style prefixes to the tableau entries. To take advantage of these styles you need to have at least rudimentary knowledge of TikZ-styling. The examples in this manual give a good indication of what is possible. For anything more exotic, please consult the TikZ manual.

The simplest way to change the style of an entry in a tableau is to add a $*$-prefix to the entry:




```
\Tableau{ 1*2*3*4*5, 6*789, {10}*{11}{12}, {{13}} }
```
2A.4

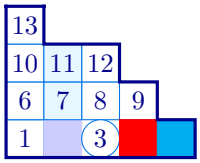
By default, the ∗-entries are given the TikZ-styling `fill=` . You can change the default ∗-style using the star style key:




```
\Tableau[star style={fill=red!20,draw=red,thick}]
        { 1*2*3*4*5, 6*789, {10}*{11}{12}, {{13}} }
```
2A.5

Note that the ∗-styling overrides the default styling of the surrounding boxes. This is because the boxes with the default styling are placed first, in the order that they are entered, followed by boxes with custom styles are placed.

The ∗-syntax is a quick shorthand for adding emphasis to some boxes in a tableau, but each of these entries will be given the same style. It is possible to give every box in the tableau different styles by putting TikZ-styling specifications inside square brackets, `[...]` , before the corresponding letter in the tableau specification. You can mix the ∗-syntax and the `[...]` -style syntax for different entries, although only one of these style settings can be applied to any given box.




```
\Tableau[french]{
    1 [blue!20]2 [circle]3 [red]4 [cyan]5,
    6 *7 8 9,
    {10} *{11} {12},
    {{13}} }
```
2A.6

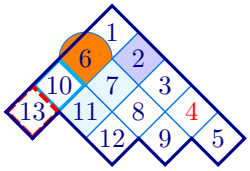
It is not necessary to add spaces between the entries in the way that this example does. This is done only to make it clearer how the style specifications are applied to the different entries in the tableau.

Omitting some technicalities, each box in a tableau is constructed as a TikZ node, so any TikZ-styling specifications for a node can be used. This example shows that some care is needed when changing the style because simply giving a colour changes both the colour used to fill the box and the text colour, which is why some of tableau entries in the last example appear to be blank. If you are already familiar with TikZ then you probably already know what to do. If not, then here is a short list of useful style specifications for TikZ nodes:

Style	Meaning
`draw` = ⟨colour⟩	Sets the boundary colour of the node
`fill` = ⟨colour⟩	Sets the fill, or background, colour of the node
`font` = ⟨font commands⟩	Sets the font
`text` = ⟨colour⟩	Sets the text colour in the node

In the style settings, any valid LaTeX colour name can be used; see, for example, the `xcolor` manual. In addition, the style specifications `thin` , `thick` , `very thick` , `ultra thick` , `dashed` , `dotted` , ... change the border thickness, and its properties. Subsection 2A.3 below describes how to use these options to customise the tableau style.

If you want to apply "complicated" style settings, such as `draw=cyan,ultra thick` , which consists of a comma-separated list of TikZ−style settings, then the entire style must be put inside matching square brackets and curly braces `[{...}]` . This is necessary to ensure that LaTeX does not get confused by the commas in the style setting (since commas are also used to separate the rows of the tableau).

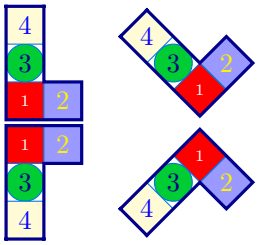


2A.7
```
\Tableau[australian]{
    1 [fill=blue!20]2 3 [text=red]4 5,
    [{circle,fill=orange}]6 *7 8 9,
    [{draw=cyan,ultra thick}]{10} *{11} {12},
    [{dashed,draw=red,ultra thick}]{13} }
```

The last example shows that it is not necessary to double-brace singleton entries when they have a style specification.

This example shows that modifying the borders of a box is problematic because boxes placed later are likely to overwrite the border (see Subsection 2A.5). As a result, modifying the borders of the boxes is not a good way to add emphasis to a tableau, so use this sparingly. In addition, the orange circle in this example looks too big, but it is what we asked for because the diameter of the circle is the horizontal width of the box. The next example shows that we get a better result for *Australian* and *Ukrainian* tableaux if we add a *minimum size* specification.

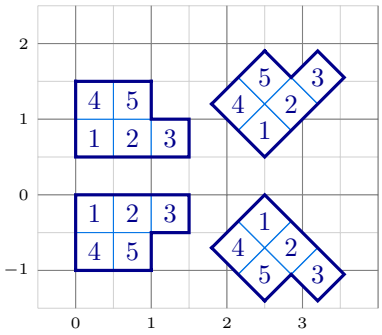
For complex or frequently used styles, we recommend using the `\tikzset` command to define the style outside of the `\Tableau` command. This makes the style both easier to read and easier to change.



2A.8
```
\tikzset{
   B/.style={fill=blue!40, text=yellow},
   G/.style={fill=green!80!blue,circle,minimum size=5mm},
   R/.style={fill=red,text=white, font=\tiny},
   Y/.style={fill=yellow!20, text=blue}
}
\Tableau[french]{[R]1[B]2,[G]3,[Y]4}
\Tableau[ukrainian]{[R]1[B]2,[G]3,[Y]4}
\Tableau[english]{[R]1[B]2,[G]3,[Y]4}
\Tableau[australian]{[R]1[B]2,[G]3,[Y]4}
```

In this example, the TikZ-styles B, G R, and Y are applied to the tableaux entries labelled 1, 2, 3, and 4, respectively. Note the use of *minimum size* to control the size of the circle. The colours used here, and throughout this manual, are for demonstration purposes only. I recommend against such garish choices in any self-respecting document!

2A.2. *Tableau coordinates.* The (x, y)-coordinates are required if, and only if, the `\Tableau` command is used inside a `tikzpicture` environment, in which case (x, y) gives the coordinates of the "outside corner" of the box in row 1 and column 1 of the tableau. The following example shows how tableaux are placed using (x, y)-coordinates inside a `tikzpicture` environment. The example also shows that, by default, the tableau boxes are square and half a unit wide.



2A.9
```
\begin{tikzpicture}[add grid]
  \Tableau(0,0)     [english]   {123,45}
  \Tableau(0,0.5)   [french]    {123,45}
  \Tableau(2.5,0.5)[ukrainian] {123,45}
  \Tableau(2.5,0)   [australian]{123,45}
\end{tikzpicture}
```

2A.3. *Tableau keys.* The optional ∗-styles and [. . .]-styles are the main mechanism for styling the individual boxes in a tableau. Using a key-value syntax, you can change aspects of the tableaux produced by `\Tableau` commands. The rest of this section describes these keys, their default values, and gives examples of how to use them.

The options below can be applied to the `\Tableau` command by giving them as a comma-separated list inside square brackets:

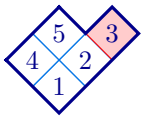
`\Tableau [options] {tableau entries}` or `\Tableau (x,y) [options] {tableau entries}`.

The options can appear in any order, with later options having precedence over earlier ones.

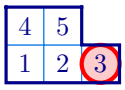
When used inside a `\Tableau` command, the options only affect that particular tableau. Most of these options, or settings, can also be used in the `\aTabset` command, or as optional arguments in `\usepackage[..]{atableau}`, to change the default settings of all tableaux in the same LaTeX group. For example, if you put the command `\aTabset[ukrainian]` in the preamble of your document then, by default, all subsequent tableaux and Young diagrams will be drawn using the Ukrainian convention.

english (default: english)
french
ukrainian
australian

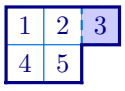
These four (mutually exclusive) options change the convention, or orientation, that is used when drawing the tableaux. Rather than defining these conventions precisely, we use the tired and true method in combinatorics of defining by example.



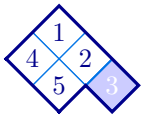
2A.10
```
\tikzset{R/.style={fill=red!20,draw=red}}
\Tableau[ukrainian]{12[R]3,45}
```



2A.11
```
\tikzset{R/.style={fill=red!20, circle, draw=red, ultra thick}}
\Tableau[french]{12[R]3,45}
```



2A.12
```
\tikzset{B/.style={fill=blue!20, dashed, thick}}
\Tableau[english]{12[B]3,45}
```

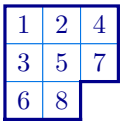


2A.13
```
\tikzset{B/.style={fill=blue!20, thick, text=white}}
\Tableau[australian]{12[B]3,45}
```

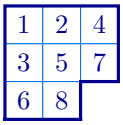
By default, the english convention is used to draw tableaux and Young diagrams. The default convention can be changed using `\aTabset`. Capitalised names, `Australian`, `English`, `French` and `Ukrainian`, are also recognised. If, for example, your document is only going to draw french tableaux and diagrams, then you can specify this when you load the package using `\usepackage[french]{atableau}`, or by adding `\aTabset{french}` to your document preamble.

align (default: centre) [accepts: centre/bottom/top]

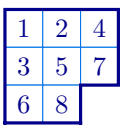
By default, the baseline of a tableau is its centre. This can be changed using the align.

Some  words

2A.14
```
Some \Tableau[align=centre]{124,357,68} words % the
default
```

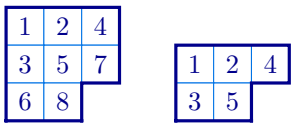
Some  words

2A.15
```
Some \Tableau[align=top]{124,357,68} words
```

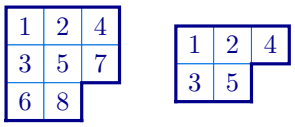
Some  words

2A.16
```
Some \Tableau[align=bottom]{124,357,68} words
```

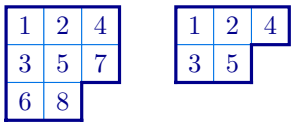
Similarly, use align to control how tableaux are aligned in displayed equations.



2A.17
```
\aTabset{align=bottom}
\Tableau{124,357,68}
\Tableau{124,35}
```



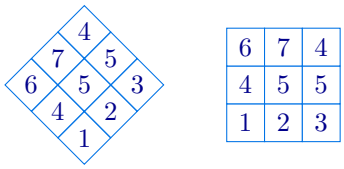
2A.18
```
% by default: align=centre
\Tableau{124,357,68}
\Tableau{124,35}
```



2A.19
```
\aTabset{align=top}
\Tableau{124,357,68}
\Tableau{124,35}
```

border (default: true) [accepts: true/false]
no border (default: false) [accepts: false/true]

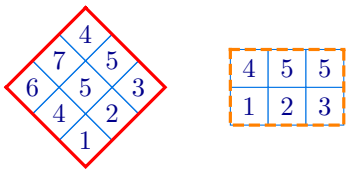
The border and no border options enable and disable the drawing of the (thick) border wall around a tableau. As with other boolean keys, border is equivalent to border=true and no border=false. The no border key is inverse to border, so no border is equivalent to no border=true, and to border=false.



2A.20
```
\Tableau[ukrainian, border=false]{123,455,674}
\Tableau[french, no border]{123,455,674}
```

border colour (default:) [accepts: any colour]
border style (default: thick,line cap=rect) [accepts: TikZ-styling]

The border colour option sets the colour of the border wall, whereas border style sets the TikZ-styling of the border wall. (So, border style can override border colour.)

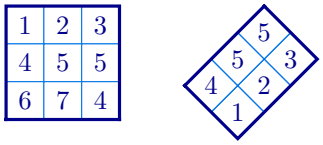


2A.21
```
\Tableau[ukrainian, border colour=red]{123,455,674}
\Tableau[french, border style={dashed,orange}]{123,455}
```

The border style key *appends* any TikZ-styles to the current styling of the wall.

box height (default: 0.5) [accepts: a decimal giving the height in cm]
box width (default: 0.5) [accepts: a decimal giving the width in cm]

Sets the width and height of the boxes in the tableau. The default height and width of the boxes is 0.5 cm when using the english and french conventions and 0.7012 cm, which is approximately $1/\sqrt{2}$cm, when using the ukrainian and australian conventions. (In all cases, the side length of the squares is 0.5 cm.)

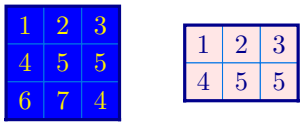


2A.22
```
\aTabset{align=top}
\Tableau[box height=0.5]{123,455,674}
\Tableau[box width=0.2,ukrainian]{123,455}
```

See also the scale, xscale and yscale options.

box fill (default: none) [accepts: any colour]

The box fill option sets the background colour of a box in a tableau, just like the fill key for a TikZ-node. If you use a dark fill colour, then you will almost certainly want to change the text colour using the text option – and you may also want to change the colours of the border walls and inner walls using border colour and inner colour, respectively.

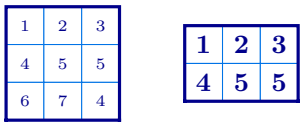


2A.23
```
\Tableau[box fill=blue,box text=yellow]{123,455,674}

\Tableau[box fill=red!10]{123,455}
```

box font [accepts:]

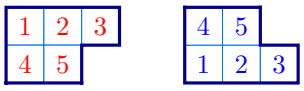
The box font key sets the font used to type the entries of a tableau. By default, tableau entries are typeset as mathematics, so this is mainly useful for changing the font size (because, for example `\bfseries $1$` does not make the 1 bold). It is only when you are using text boxes that font commands like `\itshape` and `\bfseries` will have any effect.



2A.24
```
\Tableau[box font=\tiny]{123,455,674}
\Tableau[text boxes, box font=\bfseries]{123,455}
```

box text (default:) [accepts: any colour]

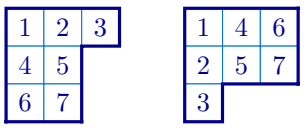
As for a TikZ-nodes, use box text to set the default text colour for the entries of the tableau boxes.



2A.25
```
\Tableau[box text=red]{123,45}
\Tableau[box text=blue, french]{123,45}
```

conjugate (default: false) [accepts: false/true]

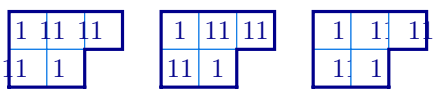
Draws the *conjugate* tableau, where the rows and columns are swapped.



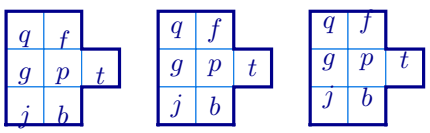
2A.26
```
\Tableau{123,45,67}
\Tableau[conjugate]{123,45,67}
```

halign (default: centre) [accepts: centre,left,right]
valign (default: centre) [accepts: bottom,centre,top]

By default, the entries in tableaux are centered, both horizontally and vertically, and it is your responsibility to ensure that the entries fit inside the tableau boxes (you can change the box dimensions using the box height, box width and scale keys). The halign and valign keys provide a crude way to change the horizontal and vertical alignment of the box entries.



2A.27
```
\Tableau[halign=left]{1{11}{11},{11}1}
\Tableau[halign=centre]{1{11}{11},{11}1}
\Tableau[halign=right]{1{11}{11},{11}1}
```

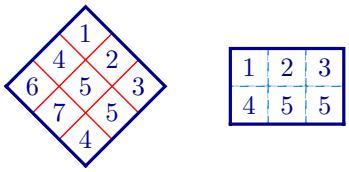


2A.28
```
\Tableau[valign=bottom]{qf,gpt,jb}
\Tableau[valign=centre]{qf,gpt,jb}
\Tableau[valign=top]{qf,gpt,jb}
```

These keys are provided for completeness only. We recommend not using them!

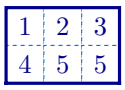
inner colour (default:) [accepts: any colour]
inner style (default: thin,line cap=rect) [accepts: TikZ-styling]

The inner colour option sets the colour of the inner wall, whereas inner style sets the TikZ-styling of the inner wall. (In particular, inner style can be used to override inner colour.)



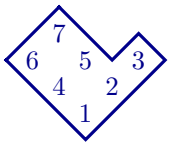
2A.29
```
\Tableau[australian, inner colour=red]{123,455,674}
\Tableau[inner style=dashed]{123,455}
```

If you closely at the second tableau in Example 2A.29, you will notice that the dashes are not very clean. This is because, for example, the dashes on bottom of one box do not properly match up with the dashes on the top of the adjacent box below it. Rather than using inner style=dashes it is better to use something like inner style={dash pattern.



2A.30
```
\Tableau[inner style={dash pattern=on 1pt off 2pt}]
  {123,455}
```

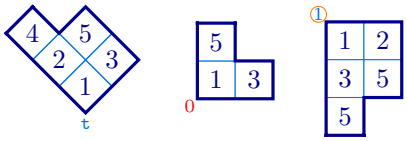
Use inner colour=none, to remove the inner tableau walls.



2A.31
```
\Tableau[ukrainian, inner colour=none]{123,45,67}
```

label [accepts: any character/text]
label style (default: font=\scriptsize,text=) [accepts: TikZ commands]

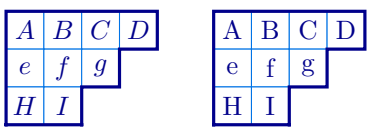
The label key adds a label to a tableau next to the $(1,1)$-box. The style of the label can be changed using the label style key.



2A.32
```
\Tableau[ukrainian, label={$\mathtt{t}$}]{13,25,4}
\Tableau[french,label=0,label style={red}]{13,5}
\Tableau[label style={draw=orange,circle,inner sep=0pt},
  label=1] {12,35,5}
```

math boxes (default: true) [accepts: true/false]
text boxes (default: false) [accepts: true/false]

Use the math boxes and text boxes to have the tableau entries typeset as either mathematics or text, respectively. By default, the entries of a tableau are typeset as mathematics.



2A.33
```
\Tableau[math boxes]{ABCD, efg, HI}      % the default
\Tableau[text boxes]{ABCD, efg, HI}
```

name (default: B) [accepts: string]

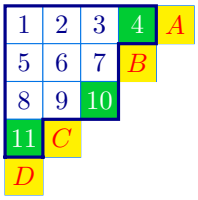
By default, the boxes can be referenced using the node names (B-1-1), (B-1-2), (B-1-3), ..., with (B-r-c) referring to the box in row r and column c. This feature is most useful when you have several tableaux inside a `tikzpicture` environment because if you only have one tableau, then it is unlikely that you need to change the default prefix for the node names. See (1.1) for a description of the anchor names, as well as Example 1.5 and Example 2A.49 for examples using node names and anchors.

ribbons [accepts: list of ribbon specifications]
snobs [accepts: list of ribbon specifications]
ribbon style (default: draw=,thin) [accepts: TikZ-styling commands]

Use the ribbons key to add a comma-separated list of ribbons to a tableau. Use snobs to add ribbons to the tableau that are placed *after* the border of the diagram is drawn. (Writing *ribbons* backwards gives *snob*bir.) The ribbon style key controls the style of the ribbons.

When adding ribbons, or snobs, you first specify the row and column indices of the *head* of the ribbon, which is the unique node of maximal content in the ribbon, and then give a sequence of r's and c's depending on whether the ribbon moves along a row or column. As always, you have the option of adding style — and you can also specify the contents of each box in the ribbon. For more details about the ribbon specifications, with examples, see Section 2F, which describes the `\RibbonTableau` command.

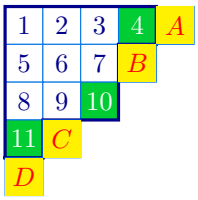
Use the ribbon box option to change the default style of the boxes in a ribbon. For example, noting that a box is a ribbon of length 1, the following code uses ribbons of length one (that is, nodes), to highlight the addable nodes of this tableau.



2A.34
```
\Tableau[star style={fill=green!80!blue,text=white},
  ribbon box={fill=yellow,text=red},
  ribbons={15_A,24_B,42_C,51_D},
]{123*4,567,89*{10},*{11}}
```

As this example indicates, ribbons are assumed to be contained inside the diagram of the tableau, so they do not change the border of a tableau.

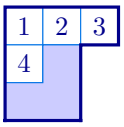
The default style for a `snobs` ribbon is given by ribbon style.



2A.35
```
\Tableau[star style={fill=green!80!blue,text=white},
  ribbon box={fill=yellow, text=red},
  snobs={15_A,24_B,42_C,51_D},
]{123*4,567,89*{10},*{11}}
```

The difference between these two pictures is that in the second picture the nodes in the snobs are drawn over the tableau border, causing the border to disappear for these nodes. For this reason, in most cases you should use ribbons, and avoid snobs.

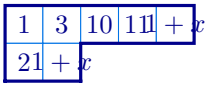
Finally, the ribbon style changes the default style of every ribbon.



2A.36
```
\Tableau[ribbon style={fill=blue!20},
  ribbons={22rc}]{123,45,67}
```

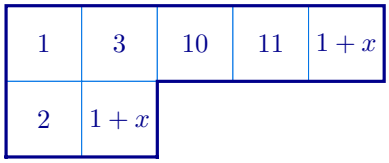
scale (default: 1) [accepts: a decimal number]
xscale (default: 1) [accepts: a decimal number]
yscale (default: 1) [accepts: a decimal number]

By design, boxes in tableaux do not automatically resize to fit their contents. For example, consider:

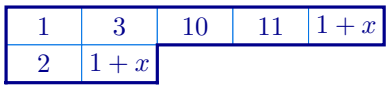


2A.37
```
\Tableau{ 13{10}{11}{1+x}, 2{1+x} }
```

This is unlikely to be the desired output! The options scale, xscale and yscale can be used to rescale the boxes in tableaux and diagrams. The names are slightly misleading because xscale rescales in the direction of increasing column index and yscale rescales in the direction of increasing row index.

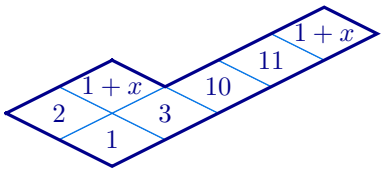



```
\Tableau[scale=2]{ 13{10}{11}{1+x}, 2{1+x} }
```
2A.38

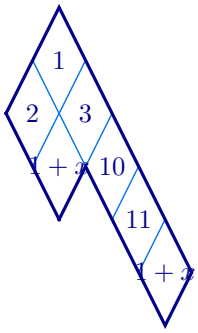



```
\Tableau[xscale=2]{ 13{10}{11}{1+x}, 2{1+x} }
```
2A.39

When using the australian and ukrainian conventions, xscale and yscale scale in both the x direction and the y-direction, reflecting how these tableaux grow with increasing column and row index, respectively.




```
\Tableau[ukrainian, xscale=2]{ 13{10}{11}{1+x}, 2{1+x} }
```
2A.40

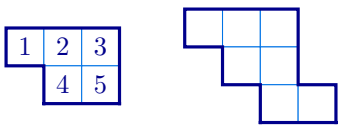



```
\Tableau[australian,yscale=2]{ 13{10}{11}{1+x},2{1+x} }
```
2A.41

The options, scale, xscale, and yscale, affect all aTableau diagrams, including tableaux, Young diagrams and abacuses. If you want to use `\aTabset` to change the default sizes of the diagrams and tableaux in your document, then it is probably better to use the box height and box width options.

shifted (default: false) [accepts: true/false]

Set to true for a shifted tableau or shifted diagram. Shifted tableaux, and shifted diagrams, are discussed in more detail in Section 2D, which describes the `\ShiftedDiagram` and `\ShiftedTableau` commands.

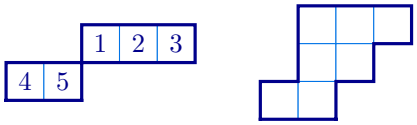



```
\Tableau[shifted]{123,45}
\Diagram[shifted]{3,2^2}
```
2A.42

See Section 2D for the options specific to shifted tableaux and shifted diagrams.

skew (default: (0)) [accepts: a partition]

Specify the skew shape for a skew tableau or skew diagram. Skew tableaux, and skew diagrams, are discussed in more detail in Section 2C, which describes the `\SkewDiagram` and `\SkewTableau` commands.

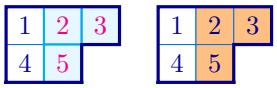



```
\Tableau[skew={2}]{123,45}
\Diagram[skew={2^2,1}]{3,2^2}
```
2A.43

See Section 2C for the options specific to skew tableaux and skew diagrams.

star style (default: fill=) [accepts: TikZ-styling]

Use the star style to add to the style of the starred entries of a tableau. (In TikZ parlance, star style *appends* appends these styles to the default star style.)

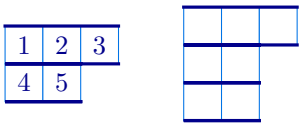


2A.44
```
\Tableau[star style={text=magenta,
   draw=cyan, thick}]{1*2*3,4*5}
\Tableau[star style={fill=orange!50}]{1*2*3,4*5}
```

Note that the tableau border is drawn *after* the star style is applied. You can use snobs to draw on top of the border.

tabloid (default: false) [accepts: true/false]

Use the tabloid key to draw a tabloid. This key is discussed in more detail in Section 2E, which describes the `\Tabloid` command.

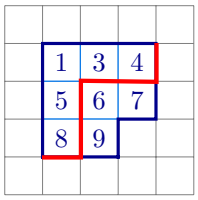


2A.45
```
\Tableau[tabloid]{123,45}
\Diagram[tabloid]{3,2^2}
```

See Section 2E for the options specific to tabloids.

tikz before [accepts: TikZ commands]
tikz after [accepts: TikZ commands]

These keys inject TikZ code into the underlying `tikzpicture` environment, where the tableau is constructed, before and after the `\Tableau` command, respectively.



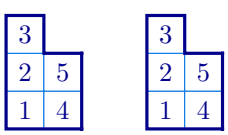
2A.46
```
\Tableau[
 tikz before={\draw[help lines,step=0.5](-0.5,-2)grid(2,0.5);},
 tikz after={\draw[ultra thick,red]
               (B-1-3.north east)--(B-1-3.south east)
               --(B-2-1.north east)--(B-3-1.south east)
               --(B-3-1.south west);
 }]{134,567,89}
```

Both before and after hooks are provided because the order in which you place TikZ commands can affect the drawing, primarily because later commands are drawn over the top of earlier ones. Note that the *before* code cannot use the named nodes B-r-c because this code is executed before these nodes are constructed, which happens when the tableau is drawn. For more examples, see Subsection 2A.5.

tikzpicture [accepts: TikZ-keys]

This command adds an optional argument to the underlying `tikzpicture` environment, which contains the tableau. The tikzpicture option is ignored when the `\Tableau` command is equipped with (x, y)-coordinates.

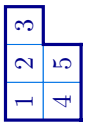
As a first example, we can use `tikzpicture={rotate=30}` to rotate a tableau by 90 degrees:



2A.47
```
\Tableau[tikzpicture={rotate=90}]{123,45}
\begin{tikzpicture}[rotate=90]
 \Tableau(0,0){123,45}
\end{tikzpicture}
```

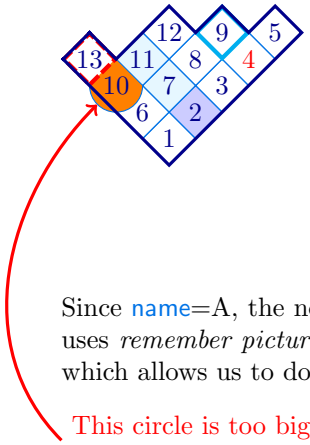
The two tableaux constructions in this example are equivalent. An oddity with both of these examples is that the numbers in the tableaux have not been rotated, which is because the TikZ *rotate* key does not

affect component objects, such as nodes, inside a TikZ picture. You can rotate the entire picture using the *transform canvas* key:




```
\Tableau[tikzpicture={transform canvas={rotate=90}}]{123,45}
```
2A.48

A more interesting example is that by using tikzpicture=remember picture we can add annotations to a tableau. For example, consider:



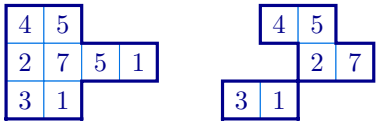

```
\Tableau[ukrainian, name=A,
  tikzpicture={remember picture}]
  { 1[fill=blue!20]23[text=red]45,
    6*78[{draw=cyan,ultra thick}]9,
    [{circle,fill=orange}]{10}*{11}{12},
    [{dashed,draw=red,ultra thick}]{13}
  }
```
2A.49

Since name=A, the nodes in this tableau are referenced as (A-1-1), (A-1-2), (A-1-3), ... As the example uses *remember picture*, other `tikzpicture` environments can reference the node names in last tableau, which allows us to do things like this:

This circle is too big!
Use minimum size=5mm

```
\tikz[remember picture, overlay]
 \draw[very thick, ->,red]
   (0,0) node[right,align=left]
      {This circle is too big!\\Use minimum size=5mm}
   to [out=135, in=225](A-3-1);
```
2A.50

2A.4. *Compositions.* As we have defined them, tableaux are always of *partition* shape, in the sense that the lengths of the rows is a weakly decreasing sequence. In fact, the `\Tableau` command also accepts tableaux of *composition* shape, where the lengths of the rows are not necessarily in decreasing order:




```
\Tableau{ 45,2751,31 }
\SkewTableau{1,2}{ 45,27,31 }
```
2A.51

Even though it is possible to draw compositions, and tableaux of composition shape, compositions are not supported in the sense that many of the options/keys assume that the diagrams are of partition shape. In addition, some options assume that tableaux and diagrams do not contain empty rows, and that the diagrams are *connected*. (For disconnected tableaux and diagrams, see the `\Multidiagram` and `\Multitableau` commands in Section 2G.)

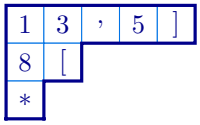
2A.5. *Drawing order.* When TikZ constructs a picture, later objects are drawn over the top of earlier ones, which can obscure earlier features. Consequently, for more complicated diagrams and tableaux it helps to know the order in which the different parts of these diagrams are drawn, which is the following:

- First, the tableau entries that use the default styling are placed, in the order that they appear in the *tableau specifications*.
- Secondly, the styled tableau entries are placed, in the order that they appear in the *tableau specifications*.
- Next, the nodes in any ribbons are placed, again in the order that the ribbons are listed. For each ribbon, first the border of the ribbon is drawn, together with any styling for the ribbon, and then the boxes in the ribbon, together with any styling, are added.

- When enabled by skew border, the skew border is drawn, after which the border of the tableau is drawn.
- The dotted rows and dotted columns keys are applied to replace the specified rows and columns with dots.
- Finally, any snobs are drawn, in the order that they are listed.

All of the tableau and Young diagram commands use this drawing order.

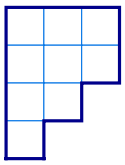
2A.6. *Special characters.* The characters [], ,, and * have special meanings in the *tableau specifications*. Enclose these characters in braces when you want to use these characters as entries in a tableau:




```
\Tableau{ 13{,}5], 8{[}, {{*}} }
```
2A.52

There is no need to enclose `]` in braces because it only becomes special when it is preceded by a matching `[` character.

2B. **Young diagrams.** A *Young diagram*, or simply a *diagram*, is an unlabelled tableau. Diagrams can be drawn using the `\Tableau` command:

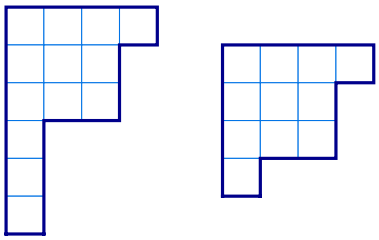


```
\Tableau{~~~,~~~,~~,~}
```
2B.1

This approach works, but it is cumbersome, hard to proofread, and easy to miscount. For this reason, aTableau provides the `\Diagram` command. The `\Diagram` command uses almost the same syntax as the `\Tableau` command:

```
\Diagram (x,y) [options] {partition}
```

As with the `\Tableau` command, the (x, y)-coordinates are needed if, and only if, the diagram is inside a `tikzpicture` environment. The `\Diagram` command allows the *partition* to be specified as either a comma-separated list of *weakly decreasing positive integers*, or by using *exponential notation*:

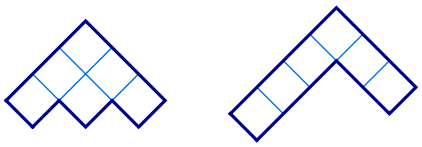


```
\Diagram{4,3^2,1^3}
\Diagram{4,3,3,1}
```
2B.2

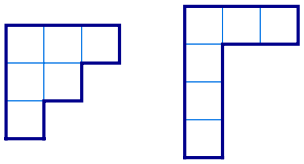
Internally, `\Diagram` actually does something like the first example in this section, so all of the options for the `\Tableau` command can be used with `\Diagram`. Rather than repeating all of the `\Tableau` options in this section, we highlight the more interesting options together with some new, diagram specific, options.

Just like the `\Tableau` command, `\Diagram` supports the four different conventions for diagrams, with english being the default:

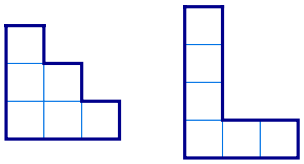
english
french
ukrainian
australian (default: english)



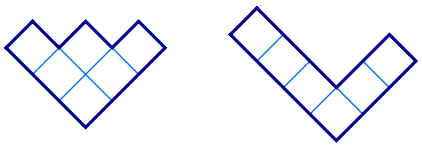
2B.3
```
\Diagram[australian]{3,2,1}
\Diagram[australian]{3,1^3}
```



2B.4
```
% English is the default convention
\Diagram[english]{3,2,1}
\Diagram{3,1^3}
```



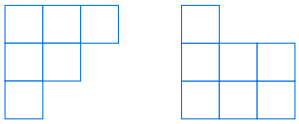
2B.5
```
\Diagram[french]{3,2,1}
\Diagram[french]{3,1^3}
```



2B.6
```
\Diagram[ukrainian]{3,2,1}
\Diagram[ukrainian]{3,1^3}
```

border (default: true) [accepts: true/false]
no border (default: false) [accepts: false/true]

If border is true, which it is by default, then the border of the tableau is drawn. If border is false, then the border of the tableau is not drawn. Notice that the border refers only to the outside border of the diagram, and not to the internal borders of the boxes on the diagram, which can be disabled using no boxes.

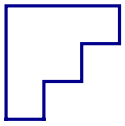


2B.7
```
\Diagram[border=false]{3,2,1}
\Diagram[no border, french]{3^2,1}
```

The no border option, and similar options, are provided only for completeness because it is the inverse of border. That is, no border is equivalent to border=false, and no border=false is equivalent to border=true.

boxes (default: true) [accepts: truefalse]
no boxes (default: false) [accepts: false/true]

The boxes and no boxes keys control whether or not the internal wall around the boxes are drawn, with these two keys being inverses of each other. By default, boxes is true, so the internal boxes are drawn. When no boxes is in force, the internal walls around boxes are not drawn.

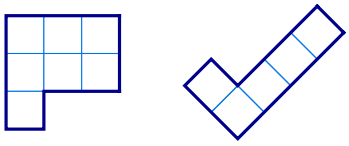


4 5
1 2 3

2B.8
```
\Diagram[no boxes]{3,2,1}
\Tableau[french, no boxes, no border]{123,45}
```

conjugate (default: false) [accepts: false/true]

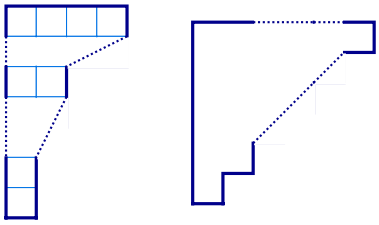
Draws the conjugate diagram, which has the rows and columns interchanged.



2B.9
```
\Diagram[conjugate]{3,2,2}
\Diagram[ukrainian, conjugate]{2,1^3}
```

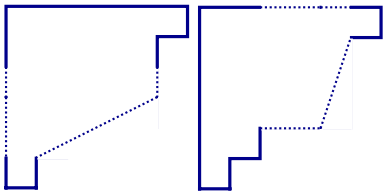
dotted columns [accepts: comma separated list of columns]
dotted rows [accepts: comma separated list of rows]

The dotted rows and dotted columns keys can be used to draw diagrams where the specified rows and columns are replaced with dots. This makes it possible to draw *generic* diagrams, where the number of rows and columns is not fully specified. Instead of dotted columns, you can use the equivalent short-hand key dotted cols.



2B.10
```
\aTabset{scale=0.8}
\Diagram[dotted rows={2,4,5}]{4^2,2^2,1^3}
\Diagram[dotted columns={5,3,4},
  no boxes] {6,5,4,3,2,1}
```

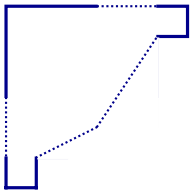
As shown above, when row and columns indices are in increasing *consecutive* order, then the corresponding rows or columns are replaced as a block. Non-consecutive indices are treated separately.



2B.11
```
\aTabset{scale=0.8, no boxes}
\Diagram[dotted rows={4,5,3}] {6,5,5,5,2,1}
\Diagram[dotted cols={5,3,4}] {6,5,5,5,2,1}
```

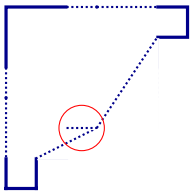
The dotted rows and dotted columns keys work by overwriting the content on the specified rows and columns, so anything that was drawn in these rows and columns will disappear. The dotted rows and dotted columns keys can be used with the `\Tableau` command, with the caveat that this will remove all boxes, with their contents, ghat are in the specified rows and columns.

When the dotted rows and dotted columns keys are used together, the dotted rows are applied first.



2B.12
```
\Diagram[scale=0.8, no boxes,
  dotted rows={4,5},
  dotted cols={4,5},
] {6,5,5,5,2,1}
```

When using these two keys together, you need to be careful when choosing which rows and columns are removed because, otherwise, artifacts can remain. Ideally, the endpoints of the rows and columns being removed should not overlap. For example, the circled horizontal dots below are a "user error", not a bug, because these dots come from trying to use dotted rows to remove row three:

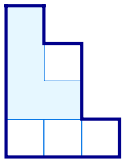


2B.13
```
\Diagram[scale=0.8, no boxes,
  dotted rows={4,5,3}, dotted cols={4,5,3},
  tikz after = {\draw[red] (1,-1.6) circle(0.3);}
] {6,5,5,5,2,1}
```

The dotted rows and dotted columns keys are not designed to be used with the conjugate key.

ribbons [accepts: ribbon specification]
snobs [accepts: ribbon specification]

Use ribbons and snobs to add ribbons to a diagram, with the snobs being added at the very end.



```
\Diagram[french, ribbons={*22*c*r*r}]{3,2^2,1}
```
2B.14

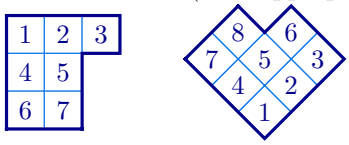
A description of the ribbon specifications, with examples, can be found in Section 2F, which describes the `\RibbonTableau` command.

2B.1. *Diagrams with entries.* The entries option provides a shortcut for drawing some common tableaux of a specified shape.

entries [accepts: contents,first,hooks,last,residues]

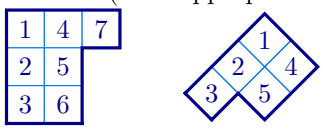
The possible choices are:

- entries=first print the *first* **standard** tableau of this shape, with respect to the Bruhat order. This is the tableau that has the numbers $1, 2 \ldots, n$ entered in order from top left to right along down successive rows (with appropriate modifications for the non-English conventions).



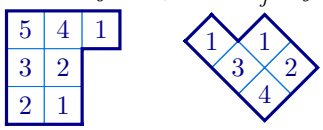
```
\Diagram[entries=first]{3,2,2}
\Diagram[ukrainian, entries=first]{3^2,2}
```
2B.15

- entries=last print the *last* **standard** tableau of this shape, with respect to the Bruhat order. This is the tableau that has the numbers $1, 2 \ldots, n$ entered in order from top to bottom down successive columns (with appropriate modifications for the non-English conventions).



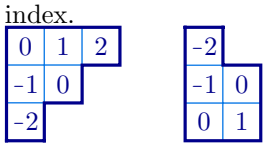
```
\Diagram[entries=last]{3,2,2}
\Diagram[australian,entries=last]{2^2,1}
```
2B.16

- entries=hooks print the diagram where each box is labelled by the corresponding *hook length*. If λ is a partition and λ' is its conjugate partition then the hook length of the box in row i and column j is $\lambda_i - i + \lambda'_j - j + 1$.



```
\Diagram[entries=hooks]{3,2,2}
\Diagram[ukrainian,entries=hooks]{2^2,1}
```
2B.17

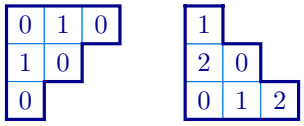
- entries=contents where each box is labelled by its *content*, which is the row index minus the column index.



```
\Diagram[entries=contents]{3,2,1}
\Diagram[french,entries=contents]{2^2,1}
```
2B.18

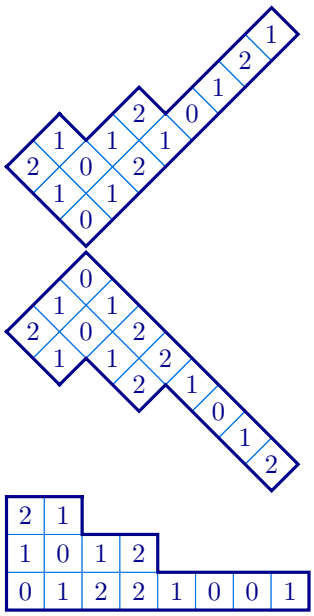
Inside a tableau, the default minus sign for a negative number looks too long, so aTableau uses `\shortminus` for negative contents. For example, it prints `\shortminus 1` instead of `-1`, which look like -1 and −1, respectively.

- entries=residues print the diagram where each box is labelled by its *residue*, which is the row index minus the column index modulo an integer $e \geq 2$, which must also be supplied.



```
\Diagram[entries=residues, e=2]{3,2,1}
\Diagram[french, entries=residues, e=3]{3,2,1}
```
2B.19

By default, residue sequences for affine type $A_e^{(1)}$, or the symmetric group and friends, are given. In addition, residues in affine types $C_e^{(1)}$, $A_{2e}^{(2)}$ and $D_e^{(2)}$, which can be accessed using cartan=C, cartan=AA, and cartan=DD, respectively.



```
% affine type C
\Diagram[ukrainian,entries=residues, e=2, cartan=C]
  {8,4,2}
```

2B.20


```
% twisted affine type A
\Diagram[australian, entries=residues, e=2,
  cartan=AA]
  {8,4,2}
```

2B.21


```
% twisted affine type D
\Diagram[french, entries=residues, e=2, cartan=DD]
  {8,4,2}
```

2B.22

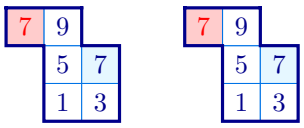
2C. **Skew tableaux and skew diagrams.** A partition μ is *contained* in another partition λ, which is written as $\mu \subseteq \lambda$, if $\mu_k \leq \lambda_k$, for $k \geq 0$. If $\mu \subseteq \lambda$ then the *skew partition* $\lambda/\mu = \{\,(r,c) \in \lambda \,|\, (r,c) \notin \mu\,\}$ is the set of nodes that are in λ and not in μ. In this manual, μ is the *inner shape* and λ/μ is the outer shape. A *skew tableau* is a labelling of the nodes in the diagram of a skew partition. Skew partitions and skew tableau can be drawn using the commands:

```
\SkewDiagram (x,y) [options] {inner shape} {outer shape}
\SkewTableau (x,y) [options] {inner shape} {skew tableau specifications}
```

As with the previous commands, the (x, y)–coordinates should be given if, and only if, the picture is inside a `tikzpicture` environment. The inner and outer shapes can either be given as a comma-separated list of non-negative integers, or in exponential notation.

Skew tableau and skew diagrams should always be connected. Use `\Multidiagram` and `\Multitableau` to draw disconnected diagrams.

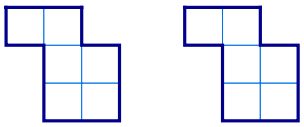
In fact, the `\SkewTableau` is just an alias for the `\Tableau`, using the skew key to set the inner shape. For this reason, all of the options for the `\Tableau` command can be used with `\SkewTableau`. The entries in a `\SkewTableau` are specified in exactly the same way as in the `\Tableau` command. In particular, the entries of skew tableaux can, optionally, be given style prefixes, both using a $*$ to add the current star style, or arbitrary TikZ-styling using `[...]`.



```
\tikzset{R/.style={fill=red!20,text=red}}
\SkewTableau[french]{1^2}{13,5*7,[R]79}
\Tableau[french,skew={1^2}]{13,5*7,[R]79}
```

2C.1

Similarly, the `\SkewDiagram` command is an alias for the `\Diagram` command, with a skew key.



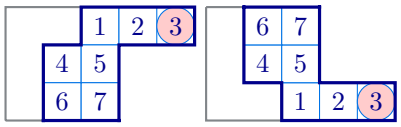
```
\SkewDiagram[french]{1^2}{2^3}
\Diagram[french,skew={1^2}]{2^3}
```

2C.2

As the `\SkewTableau` and `\SkewTableaua` commands are shortcuts, all of the options for the `\Tableau` and `\Diagram` commands can be used with the `\SkewTableau` and `\SkewDiagram` commands, respectively. In addition, the following options are supported:

skew border (default: false) [accepts: true/false]
no skew border (default: true) [accepts: false/true]

These two keys are inverse to each other. When skew border is true, the border of the (inner) skew shape is drawn.

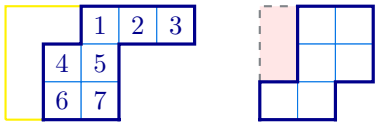



```
\tikzset{R/.style={fill=red!20,circle}}
\SkewTableau[skew border]{2,1^2}{12[R]3,45,67}
\SkewTableau[french,skew border]{2,1^2}{12[R]3,45,67}
```

Note that skew border only draws the border of the skew shape and not the boxes inside the inner skew shape. Use skew boxes to draw the interior walls of the boxes in the skew shape.

skew border style (default: draw=, fill=, thick) [accepts: TikZ-styling]

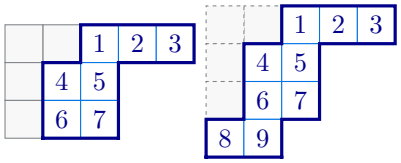
Use skew border style to change the style of the skew border. Since the skew border is only drawn when skew border is set, the skew border style does not have any effect unless skew border has been set to true.




```
\SkewTableau[skew border style={draw=yellow},
    skew border]{2,1^2}{123,45,67}
\SkewDiagram[skew border style={dashed,fill=red!10},
    skew border]{1^2}{2^3}
```

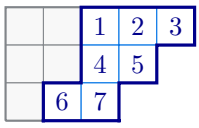
skew boxes (default: false) [accepts: true/false]
skew box style (default: thin, fill=) [accepts: TikZ-styling]

The skew boxes key draws the inner walls in the inside the inner partition of a skew shape. Use skew box style to change the default shading of these boxes.




```
\SkewTableau[skew boxes] {2,1^2}{123,45,67}
\SkewTableau[skew boxes,
  skew box style={dash pattern=on 1pt off 2pt}
] {2,1^2}{123,45,67,89}
```

Using the options skew border and the skew boxes together gives the skew (inner) shape both inner and outer borders.

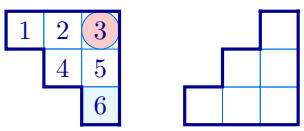



```
\SkewTableau[skew border, skew boxes] {2^2,1}{123,45,67}
```

2D. **Shifted tableaux and shifted diagrams.** *Strict partitions*, which have strictly increasing parts, and *shifted tableaux*, which are of strict partition shape, appear in several places in representation theory, such as in the study of projective representations of the symmetric groups. These tableaux and diagrams are drawn with row r shifted by $(r-1)$-units along the row, so that the first box in each row has content 0. These diagrams and tableaux can be drawn using the following commands:

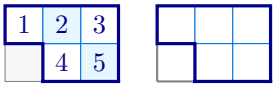
```
\ShiftedDiagram (x,y) [options] {partition }
\ShiftedTableau (x,y) [options] {tableau specification}
```

As usual, the (x, y)-coordinates are necessary if, and only if, these commands are used inside a `tikzpicture` environment. The partition in a `\ShiftedDiagram` can be given using exponential notation and the tableau specification for a `\ShiftedTableau` can include the usual optional style prefixes.




```
\tikzset{R/.style={fill=red!20,circle}}
\ShiftedTableau{12[R]3,45,*6}
\ShiftedDiagram[french]{3,2,1}
```

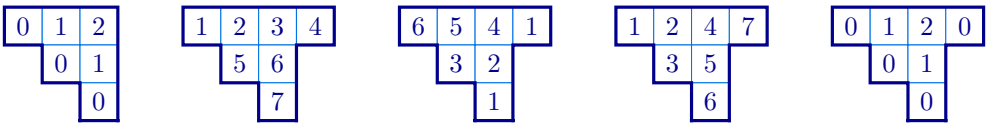
In the literature, shifted tableaux and shifted diagrams almost always have strict partition shape, however, this is not enforced by these commands. Under the hood, the `\ShiftedTableau` is the `\Tableau` command with the shifted option set to true. Similarly, `\ShiftedDiagram` is the same as using the `\Diagram` command with the shifted option. Consequently, all of the options for the `\Tableau` and `\Diagram` commands can be used with `\ShiftedTableau` and `\ShiftedDiagram`, respectively. In particular, options like skew boxes can be used to highlight the shifted part of the diagram



2D.2
```
\ShiftedTableau[skew boxes]{1*23,4*5}
\ShiftedDiagram[skew border]{3,2}
```

The entries key works as expected for shifted diagrams:

2D.3
```
\ShiftedDiagram[entries=contents]{3,2,1}
\ShiftedDiagram[entries=first]{4,2,1}
\ShiftedDiagram[entries=hooks]{4,2,1}
\ShiftedDiagram[entries=last]{4,2,1}
\ShiftedDiagram[entries=residues,e=3]{4,2,1}
```

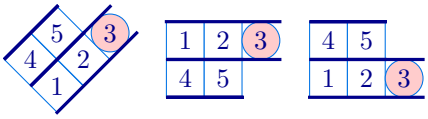


The definition of hook lengths for shifted diagrams can be found in the literature; for example, see [**?**]!

2E. **Tabloids.** A *tabloid* is an equivalence class of tableau, where two tableaux are equivalent if they have the same entries in each row, up to a permutation. In the literature, tabloids are usually drawn with lines above and below each row, and without side borders. Tabloids can be drawn with the `\Tabloid` command.

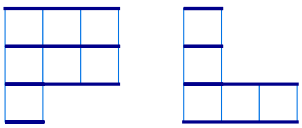
`\Tabloid (x,y) [options] {partition }`

The `\Tabloid` command functions in the same way as the `\Tableau` command, except that it draws tabloids. In fact, the `\Tabloid` is a special case of the `\Tableau` command with the tabloid option set.



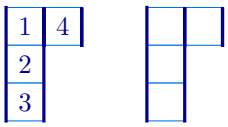
2E.1
```
\tikzset{R/.style={fill=red!20,circle,minimum size=5mm}}
\Tabloid[ukrainian]{12[R]3,45}
\Tabloid[english]  {12[R]3,45}
\Tabloid[french]   {12[R]3,45}
```

The aTableau package does not provide a dedicated command for tabloid diagrams, however, they can be drawn using the `\Diagram` command together with the tabloid option:



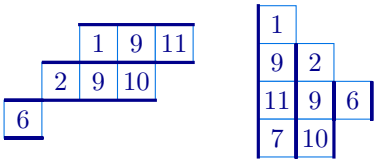
2E.2
```
\Diagram[tabloid]{3^2,1}
\Diagram[tabloid,french]{3,1^2}
```

Some authors work with *column tabloids*. Such tableaux do not have a dedicated aTableau command, however, they can be drawn using the conjugate option:



2E.3
```
\Tabloid[conjugate]{123,4}
\Diagram[conjugate,tabloid]{3,1}
```

Similarly, skew tabloids and shifted tabloids can be drawn using the skew option.



2E.4
```
\Tabloid[skew={2,1}]{19{11},29{10},6}
\Tabloid[conjugate,skew={0,1,2}]{19{11}7,29{10},6}
```

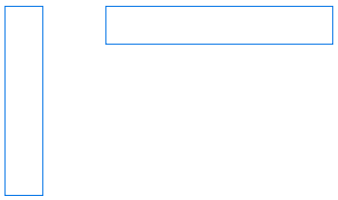
2F. **Ribbon tableaux.** A *ribbon* in a tableau, or a diagram, is a connected strip of boxes R that are totally ordered by their contents. (Recall from **??** that the content of the box in row a and column b is $b-a$.) Therefore, a ribbon does not contain a ⊞-square, and if $(a,b) \in R$ then at most one of $(a+1,b)$ and $(a,b-1)$ belongs to R. The *head* of a ribbon R is the unique node of maximal content. A *ribbon tableau* is a tableau that is tiled by ribbons. A box is a ribbon of length 1, so every tableau is a ribbon tableau.

`\RibbonTableau (x,y) [options] {ribbon specifications}`

Like the other commands, the (x,y)-coordinates are optional and are required if, and only if, the diagram is being used as part of a `tikzpicture` environment. All options for the `\Tableau` command can be used for the ribbon command, so we refer to Section 2A for a description of the available options.

2F.1. *Ribbon specifications.* Unlike the `\Tableau` command, the entries of a ribbon tableau are given by ribbon specifications (rather than tableau) specifications. Ribbon specifications are also used by the ribbons and snobs options discussed in Subsection 2A.3.

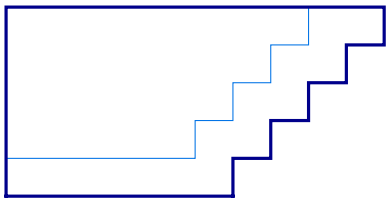
To understand the ribbon specifications, observe that if (a,b) is the head of a ribbon, then we can walk along the ribbon by specifying whether the row index increases or the column index decreases. That is, a ribbon is uniquely determined by specifying its head (a,b) together with a sequence of r's and c's to indicate when the row index increases, or the column index decreases, respectively. For example, vertical and horizontal ribbons are given by a sequences of r's and c's, respectively:



2F.1
```
\aTabset{align=top, no border}
\RibbonTableau{11rrrr}
\RibbonTableau{16ccccc}
```

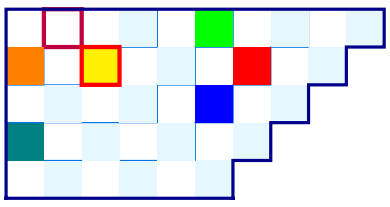
The border of a ribbon tableau is the smallest (skew) partition that contains all of its ribbons. In the last example shows, the no border key is used to disable the ribbon tableau border so that we can better see the ribbon, which is the full diagram in these two cases.

The following diagram shows a see-sawing ribbon with head in row 1 and column 10, and tail in row 5 and column 1.



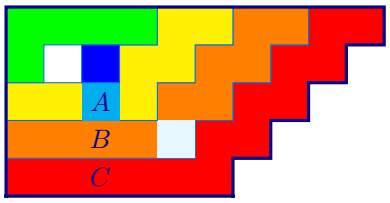
2F.2
```
\RibbonTableau{1{10}crcrcrcrccccc}
```

As with tableau entries, each box in a ribbon can be preceded with a style specification, either using `*` for the star style or using `[...]` for more customised style specifications.



2F.3
```
\RibbonTableau{
    *1{10}c*rc*rc*rc*rc*cc*cc,
    *18c[fill=red]rc[fill=blue]rc*rc*cc[fill=teal]c,
    [fill=green]16c*rc*rc*cc,
    *14c[{draw=purple,ultra thick}]cc[fill=orange]r,
    [{fill=yellow,draw=red,ultra thick}]23
}
```

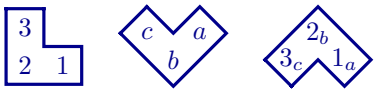
In particular, the yellow box in this example highlights that boxes are ribbons of length 1. Most of the time, you want to style the entire ribbon rather than styling the individual boxes in the ribbon. Optionally, the style for the entire ribbon can be given inside parentheses (...) at the *start* of the ribbon. Unlike the other style specifications, the ∗-shorthand for the star style cannot be applied to an entire ribbon.



2F.4
```
\RibbonTableau{
    (fill=red)1{10}crcrcrcrccc_Ccc,
    (fill=orange)18crcrc*rcc_Bcc,
    (fill=yellow)16crcr[fill=cyan]c_Acc,
    (fill=green)14cccr,
    (fill=blue)23
}
```

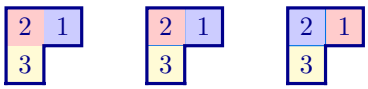
Any styles for a ribbon are applied to the region bounded by the closed path given by the border of the ribbon. The ribbon style is applied first, after which any styled boxes in ribbon are drawn. As this example shows, the optional style of any box in the ribbon takes precedence over the style of the full ribbon.

Finally, it is often necessary to label some of the boxes in the ribbon. Optionally, text can be added to the boxes in a ribbon by supplying it as a *subscript* to the corresponding entry in the ribbon specification:



2F.5
```
\RibbonTableau[french]    { 12_1 c_2 r_3 }
\RibbonTableau[ukrainian] { 12_a c_b r_c }
\RibbonTableau[australian]{ 12_{1_a} c_{2_b} r_{3_c} }
```

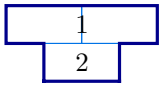
Here we have put spaces between the entries for the different boxes in the ribbon for clarity. There are often many ways to draw the same ribbon tableau, especially if the entries have styling.



2F.6
```
\tikzset{Fill/.style={fill=#1!20!white},
  B/.style={Fill=blue}, R/.style={Fill=red},
  Y/.style={Fill=yellow}}
\RibbonTableau{ [B]12_1   [R]c_2   [Y]r_3}
\RibbonTableau{ [B]12_1, [R]11_2, [Y]21_3 }
\Tableau{       [B]2     [R]1,    [Y]3 }
```

As shown, the easiest way to draw this particular tableau is using the `\Tableau` command.

There is no dedicated syntax for putting a label on the boundary between two boxes in the diagram, but this can be done using the named anchors of (1.1).



2F.7
```
\RibbonTableau[shifted, tikz after={
    \node at (B-1-2.east){$1$};
    \node at (B-2-2.east){$2$};
}]{ 14c, 12c, 23c }
```

We have now seen the full ribbon specifications. To summarise:

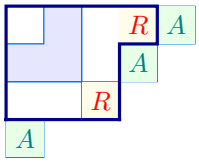
- The ribbon specification starts with optional TikZ-styling for the ribbon, enclosed in (...).
- The first entry in the ribbon is given as ab, if the head of the ribbon is in row a and column b. The remaining entries in the ribbon are specified by a sequence of r's and c's, depending on whether the next box in the ribbon is given by increasing the row index, or decreasing the column index, respectively.
- Optionally, each box in the ribbon, including the head, can be given TikZ-styling by prefixing it with a ∗, which gives the ribbon star style, or by giving TikZ-styling keys inside square brackets [...].

Finally, when drawing the border, ribbons in a `\RibbonTableau` are assumed to be contained in the smallest Young diagram that contains the ribbons. This said, it is up to you to ensure that the boxes in

the ribbon not have negative column indices. Inside a `\Diagram` or `\Tableau` command, any ribbons added using ribbons, or snobs, do not affect the border of the diagram or tableau, and it is your responsibility to ensure that the boxes in the ribbon are inside the diagram or tableau. This said, it is a useful feature to be able to place boxes outside of the diagrams, which can be done using ribbons for `\Tableau` or snobs as these do not contribute to the border of a (ribbon) tableaux.

ribbons [accepts: comma separated ribbon specifications]
snobs [accepts: comma separated ribbon specifications]

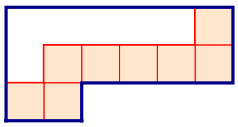
You can add ribbons and snobs to a `\RibbonTableau` in exactly the same way that you add them to `\Tableau`. At first sight, this seems unnecessary because ribbon tableaux are composed of ribbons, however, the point is that any ribbon added to a tableau using ribbons or snobs is not assumed to be inside the border of the tableau.



2F.8
```
\tikzset{A/.style={fill=green!10, text=teal},
         R/.style={fill=yellow!10, text=red}}
\RibbonTableau[
  ribbons={[A]15_A,[A]24_A,[A]41_A, [R]14_R,[R]33_R}]
  {11, (fill=blue!10)12rc, 14crrcc}
```

ribbon style (default: draw=, thin) TikZ-styling
ribbon box (default: -) TikZ-styling

Use the ribbon style and ribbon box options to append to the default styling of the ribbons and the boxes in the ribbons, respectively. By default, ribbons have an inner wall and the boxes in ribbons have no styling.

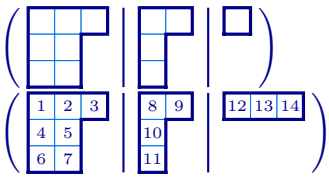


2F.9
```
\RibbonTableau[ribbon style={draw=red, thick},
    ribbon box={fill=orange!20, draw=red}]{16rccccrc}
```

2G. **Multidiagrams and multitableaux.** Multitableaux and multidiagrams are ℓ-tuples of tableaux and diagrams, respectively. These diagrams appear in many places in representation theory, such as when considering representations of wreath products or cyclotomic Hecke algebras. For convenience, aTableau provides the `\Multidiagram` and `\Multitableau` commands for drawing these diagrams, even though it is not difficult to construct such tableaux and diagrams using the `\Tableau` and `\Diagram` commands.

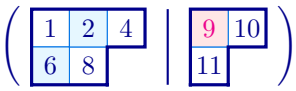
```
\Multidiagram (x,y) [options] {multipartition specifications}
\Multitableau (x,y) [options] {multitableau specifications}
```

Our preferred notation for a multipartition $\boldsymbol{\lambda}$ is to write $\boldsymbol{\lambda} = (\lambda^{(1)}|\lambda^{(2)}|\dots|\lambda^{(\ell)})$, where $\ell \geq 1$ is the *level* and the partitions $\lambda^{(1)},\dots,\lambda^{(\ell)}$ are the *components* of $\boldsymbol{\lambda}$. Similarly, a multitableau is written as $\mathsf{t} = (\mathsf{t}^{(1)}|\dots|\mathsf{t}^{(\ell)})$, where the tableaux $\mathsf{t}^{(1)},\dots,\mathsf{t}^{(\ell)}$ are the *components* of t. Accordingly, in the `\Multidiagram` and `\Multitableau` commands, the pipe symbol `|` is used to separate components:



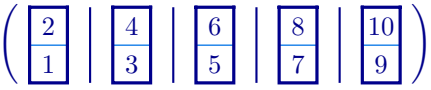
2G.1
```
\aTabset{scale=0.7}
\Multidiagram{3,2^2|2,1,1|1}
\Multitableau[box font=\tiny]
    { 123,45,67|89,{{10}},{{11}}|{12}{13}{14} }
```

As shown, the `\Multidiagram` command accepts exponential notation for multipartitions. The entries in a `\Multitableau` can be given optional styling specifications, exactly as inside the `\Tableau` command.



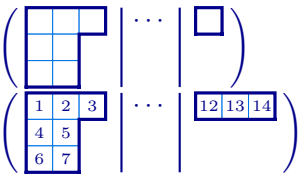
2G.2
```
\tikzset{R/.style={fill=red!10,text=magenta}}
\Multitableau{1*24,*68|[R]9{10},{{11}}}
```

Multitableaux and multidiagrams can, in principle, have arbitrary level. In practice, the level is constrained by the page width.



```
2G.3
\Multitableau[french]{1,2|3,4|5,6|7,8|9,{{10}}}
```

Both the `\Multitableau` and `\Multidiagram` commands accept a special component, `...`, that is used to add dots between the separators:

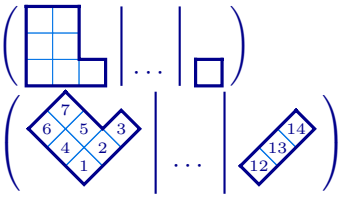


```
2G.4
\aTabset{scale=0.7}
\Multidiagram{3,2^2|...|1}
\Multitableau[box font=\tiny]
    {123,45,67|...|{12}{13}{14}}
```

The `\Multitableau` and `\Multidiagram` commands accept most of the options of the `\Tableau` and `\Diagram` commands, together with a few options that are specific to multitableaux and multidiagrams. This section describes the new options, and highlights how some of the other options are used.

english
french
ukrainian
australian (default: english)

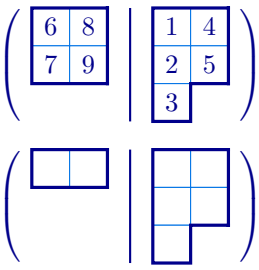
All four tableaux conventions are supported for multitableaux and multidiagrams, with the caveat that all component diagrams use the same convention.



```
2G.5
\aTabset{scale=0.7}
\Multidiagram[french]{3,2^2|...|1}
\Multitableau[ukrainian, box font=\tiny]
    {123,45,67|...|{12}{13}{14}}
```

conjugate

The conjugate option prints the conjugate mutlitableaux and multidiagrams. Conjugation for multitableaux and multidiagrams reverses the order of the components and then conjugates the component diagrams.



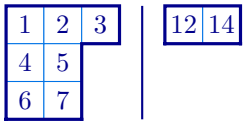
```
2G.6
\Multitableau[conjugate]{ 123,45 | 67,89}
\Multidiagram[conjugate]{ 3,2 | 1^2 }
```

delimiters (default: ()) [accepts: pair of LaTeX delimiters]
left delimiter (default: () [accepts: a LaTeX delimiter]
right delimiter (default:)) [accepts: a LaTeX delimiter]

The delimiter options control the delimiters that are put on the left and right of multitableaux and multidiagrams. These options accept any LaTeX delimiter.

```
\aTabset{scale=0.7}
\Multidiagram[left delimiter=\langle,
    right delimiter={]}]{2,1|1}
\Multidiagram[delimiters={\|}{\}} ]{1^2|2}
```

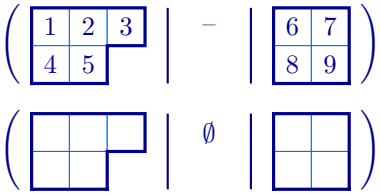
To remove a delimiter, set left delimiter or right delimiter to empty.



```
\Multitableau[left delimiter=, right delimiter=]
  {123,45,67|{12}{14}}
```

empty (default: \textendash) [accepts: any LaTeX]

The empty option determines what is printed when a component diagram is empty. By default, the en-dash symbol – is used.



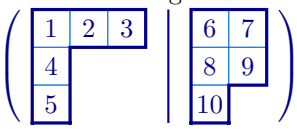
```
\Multitableau{ 123,45 | | 67,89}
\Multidiagram[empty=$\emptyset$]{ 3,2 | | 2^2 }
```

entries [accepts: contents,first,hooks,last,residues]
charge [accepts: |-separated list of charges]

Use the entries key to draw particular types of tableau of the specified shape. Use charge to provide offsets in each component when using entries=contents and entries=residues (and no charge should be set when using entries=first and entries=last).

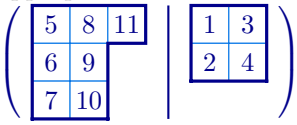
The possible choices are:

- entries=first print the *first* **standard** tableau of this shape, which has the numbers $1, 2 \ldots, n$ entered in order from top left to right along down successive rows (with appropriate modifications for the non-English conventions).



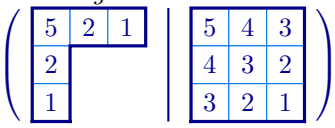
```
\Multidiagram[entries=first]{3,1^2|2^2,1}
```

- entries=last print the *last* **standard** tableau of this shape, which has the numbers $1, 2 \ldots, n$ entered in order from top to bottom down successive columns, and right to left along components (with appropriate modifications for the non-English conventions).



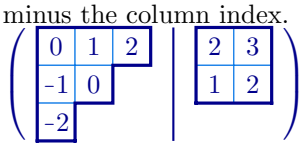
```
\Multidiagram[entries=last]{3,2,2|2^2}
```

- entries=hooks print the diagram where each box in each component is labelled by the corresponding *hook length*.



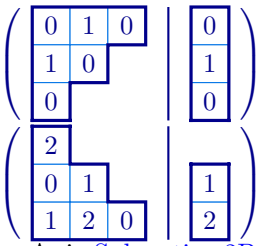
```
\Multidiagram[entries=hooks]{3,1^2|3^3}
```

- entries=contents print the diagram where each box is labelled by its *content*, which is the row index minus the column index.



```
\Multidiagram[entries=contents, charge={0,2}]
  {3,2,1|2^2}
```

- entries=residues print the diagram where each box is labelled by its *residue*, which is the row index minus the column index modulo some integer e, which must also be supplied.



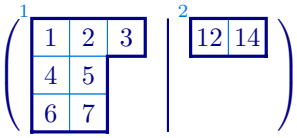
```
\Multidiagram[entries=residues, e=2]{3,2,1|1^3}
```
2G.14

```
\Multidiagram[french,entries=residues, e=3,
  charge={1,2}]{3,2,1|1^2}
```
2G.15

As in Subsection 2B.1, residues for the other affine Cartan types can be specified using cartan=??.

label [accepts: a |-separated list of labels]

Use the label key to add labels to the component diagrams.

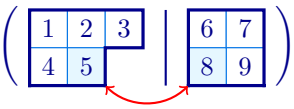


```
\Multitableau[label={1|2}]{123,45,67|{12}{14}}
```
2G.16

No extra space is allowed for the labels, so use xoffsets and yoffsets to adjust for wide labels.

name (default: B) [accepts: character]

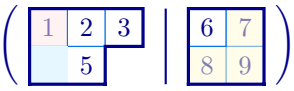
The name option is used to change the default prefix for the box node names. In a tableau, or diagram, the box names depend on the *row* and *column* indices. In a multitableau, or multidiagram, the box names depend on the *component*, *row* and *column* indices. Explicitly, by default, in multitableaux and multidiagrams the names of the take the form B-k-r-c, where k is the component index, r is the row index, and c is the column index of the box. The B can be changed like using the name key.



```
\Multitableau[name=A, tikz after = {
  \draw[red, thick,<->](A-1-2-2.south east) to
       [out=315,in=225] (A-2-2-1.south west);
  }] { 123,4*5 | 67,*89}
```
2G.17

ribbons [accepts: a |-separated list of ribbon specifications]
snobs [accepts: a |-separated list of ribbon specifications]

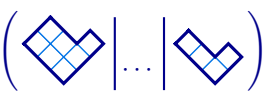
The ribbons and snobs keys work in exactly the same way for multitableaux and multidiagrams as they do for tableaux and diagrams except that the ribbons for different components are separated by a pipe |.



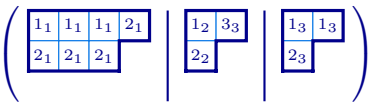
```
\tikzset{R/.style={fill=red!10, opacity=0.5},
         Y/.style={fill=yellow!20,opacity=0.5}}
\Multitableau[ribbons={(R)11*r|(Y)12rc}]
    { 123,45 | 67,89}
```
2G.18

rows [accepts: a decimal number]

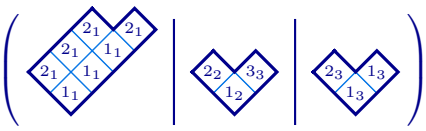
The rows option sets the number of rows covered by the delimiters in multitableaux and multidiagram. By default, the \Multitableau and \Multidiagram commands choose the size of the delimiters to match the component diagrams, which may not be what you want especially when using the australia and ukrainian conventions. The value of rows can be a decimal number.



```
\Multidiagram[ukrainian,scale=0.5]{3,2^2|...|2,1^2}
```
2G.19



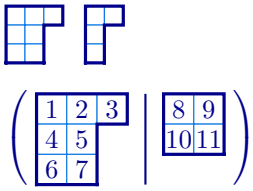
```
\Multitableau[rows=3,scale=0.8,box font=\tiny]
     {{1_1}{1_1}{1_1}{2_1},{2_1}{2_1}{2_1} |
      {1_2}{3_3},{{2_2}} |
      {1_3}{1_3},{{2_3}}}
```



```
\Multitableau[ukrainian,rows=2.5,scale=0.8,box font=\tiny]
     {{1_1}{1_1}{1_1}{2_1},{2_1}{2_1}{2_1} |
      {1_2}{3_3},{{2_2}} |
      {1_3}{1_3},{{2_3}}}
```

separators (default: true) [accepts: true/false]
no separators (default: false) [accepts: false/true]

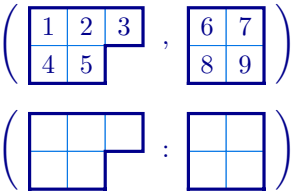
When separators is true, delimiters are drawn around the component diagrams in a multitableaux and a multidiagram and separators are drawn between the component shapes, and no separators is the inverse option. By default, separators and delimiters are drawn.



```
\Multidiagram[no separators,scale=0.5]
  {3,2^2|2,1^2}
\Multitableau[separators,scale=0.8]
  {123,45,67 | 89,{10}{11}}
```

separator (default: |) [accepts: character]

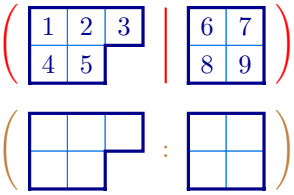
The separator determines what is printed between the component diagrams in multitableaux and multipartition. By default, separator=— draws a straight line between the components, but any character can be used.



```
\Multitableau[separator={,}]{ 123,45 | 67,89}
\Multidiagram[separator=:]{ 3,2 | 2^2 }
```

separator colour (default:) [accepts: LaTeX colour]

The separator colour sets the colour of the delimiters and the separator between component diagrams. By default, the separators are the same colour as the diagram border.

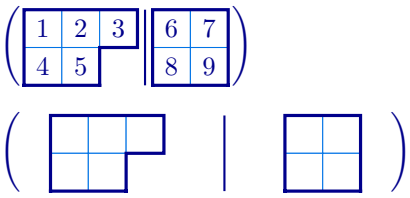


```
\Multitableau[separator colour=red]{123,45 | 67,89}

\Multidiagram[separator color=brown, separator=:]
  {3,2 | 2^2 }
```

separation (default: 0.3) [accepts: a decimal number (the distance in cm)]

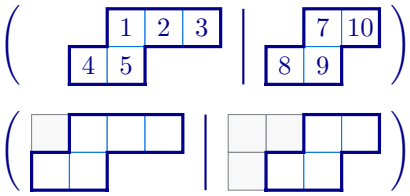
The separation key sets the distance between the separators and the component tableaux and diagrams.



```
\Multitableau[separation=0.1]{123,45 | 67,89}
\Multidiagram[separation=0.8]{3,2 | 2^2}
```


skew a |-separated list of partitions

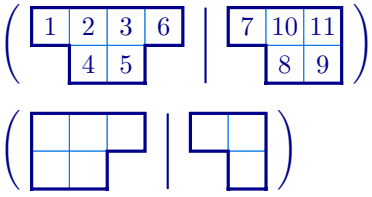
Skew multishapes can be drawn using the skew option to specify a |-separated list of inner shapes.



```
\Multitableau[skew={2,1|1}]{ 123,45 | 7{10},89}

\Multidiagram[skew={1|2,1}, skew boxes]{ 3,2 | 2^2 }
```

There is no command for drawing shifted multitableau or multidiagrams, however, such diagrams can be be drawn using an appropriate skew key. In the same way, it is possible to draw multitableaux and multidiagrams with components that are a mixture of non-skew, skew and shifted shapes.

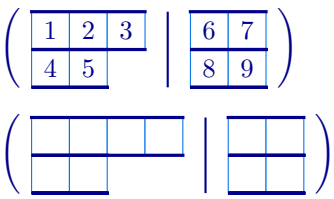


```
\Multitableau[skew={0,1|0,1}]{ 1236,45 | 7{10}{11},89}

\Multidiagram[skew={|0,1}]{ 3,2 | 2,1 }
```

tabloid

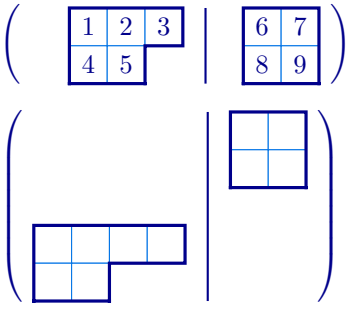
Use the tabloid key for multitabloid diagrams and tableaux.



```
\Multitableau[tabloid]{ 123,45 | 67,89}
\Multidiagram[tabloid]{ 4,2 | 2^2 }
```

xoffsets (default: 0) [accepts: comma separated list of x-offsets]
yoffsets (default: 0) [accepts: comma separated list of y-offsets]

The \Multitableau and \Multidiagram commands try to draw the diagrams as compactly as possible with their first rows aligned, subject to the value of separation. Use the xoffsets to offset the x-coordinates of the component diagrams and yoffsets to offset their y-coordinates.



```
\Multitableau[xoffsets={0.5,0.2}]{ 123,45 | 67,89}

\Multidiagram[yoffsets={0,1.5}]{ 4,2 | 2^2 }
```

When using xoffsets or yoffsets, you may want to use no separators to disable the separators between component shapes.

3. Abacuses

Gordon James introduced the abacus, with beads and runners, as another way to represent a partition [?]. The abacus is particularly useful in modular representation theory, where the number of runners is the (quantum) characteristic. Abacus combinatorics are now used to help many algebras beyond their initial appearance in the representation theory of the symmetric groups.

The partition corresponding to an abacus is given by counting the number of empty bead positions before each bead on the abacus. The `\Abacus` command, which draws an abacus representation of a partition, has a similar syntax to the tableau and diagram commands.

```
\Abacus (x,y) [options] {number of runners} {bead specifications}
```

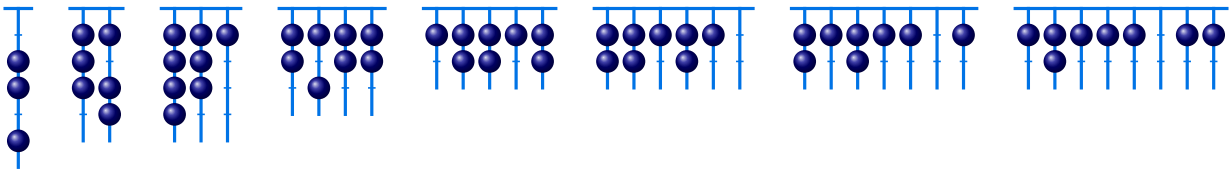
As with other aTableau commands, the (x, y)-coordinates are necessary if, and only if, the abacus is part of a `tikzpicture` environment. The number of runners is a positive integer that specifies the number of abacus runners.

3A. Bead specifications.

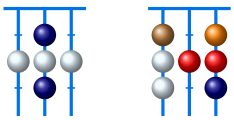
In their simplest form, the bead specifications are a comma separated list of non-negative integers specifying a partition. Partitions can be given either as comma separated lists of non-negative integers, or using exponential notation. The full bead specifications add optional styling and bead labels.

Here are some examples showing basic usage of the `\Abacus` command:

3A.1
```
\aTabset{align=top}
\Abacus{1}{2,1,1}      \quad \Abacus{2}{2,1^2,0^3}  \quad \Abacus{3}{2,1^2,0^5}  \quad
\Abacus{4}{2,1^2,0^5}  \quad \Abacus{5}{2,1^2,0^5}  \quad \Abacus{6}{2,1^2,0^5}  \quad
\Abacus{7}{2,1^2,0^5}  \quad \Abacus{8}{2,1^2,0^5}
```



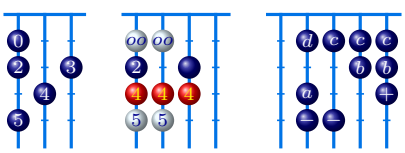
The beads in an abacus correspond to the parts of the partition. Just as with the `\Tableau` command, each of the beads can be given optional styling, which can either be given by prepending a `*`, which applies the abacus star style, or by giving TikZ styling-keys inside square brackets `[...]`. When exponential notation is used, the styles are applied to all of the associated beads.



3A.2
```
\aTabset{align=top}
\Abacus{3}{3,*2^3,1}
\Abacus{3}{2,*1,[ball color=red]1^2, *1,
  [ball color=orange]1,[ball color=brown]0}
```

In particular, the two beads corresponding to `1^2` are coloured red, whereas the other three beads that correspond to a 1 are orange and the off-white colour, which is the colour given by the abacus star style. As with the tableaux commands, more complicated bead styles can be defined using `\tikzset`.

In addition, to applying style to the individual beads on the abacus, labels for the beads can (optionally) be given as subscripts in the bead specifications. Again, the same labels are applied to all of the corresponding beads when exponential notation is used.



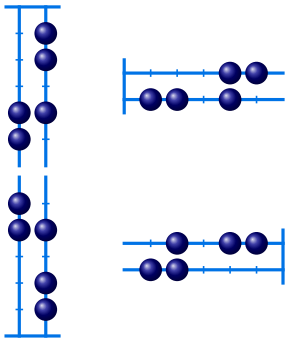
3A.3
```
\aTabset{align=top}
\tikzset{R/.style={ball color=red, text=yellow}}
\Abacus{3}{5_5, 4_4, 3_3, 2_2,0_0}
\Abacus{4}{*5^2_5, [R]4^3_4, 3, 2_2,*0^2_{oo}}
\Abacus{5}{8^2_-,7_+,5_a,4^2_b,1_c^3,1_d}
```

The bead labels are arbitrary, however, anything winder than the bead will spill into the abacus. Traditionally, the beads are unlabelled, but your can give optional labels to as many beads as you like.

3B. **Abacus keys.** The abacus keys, or options, control the general appearance of the abacuses drawn by the `\Abacus` command. Many of the options, or keys, for tableaux and diagrams can also be applied to abacuses, including align, name, scale, tikz after, tikz before, and tikzpicture. See Chapter 4 for the complete list.

south (default: true) true/false
north (default: false) false/true
east (default: false) false/true
west (default: false) false/true

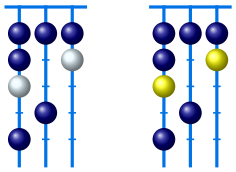
Abacuses, just like tableaux, have four different conventions. By default, abacuses are drawn in the southerly direction, from top to bottom. Using these keys, abacuses can also be drawn to the north, east and west.



3B.1
```
\Abacus[south]{2}{4^3,2,1}        % the default
\Abacus[east]{2}{4^3,2,1}
\Abacus[north]{2}{4^3,2,1}
\Abacus[west]{2}{4^3,2,1}
```

abacus star style (default: text=■,text=□) [accepts: TikZ-style keys]

The abacus star style key appends TikZ-style kleys to the ∗-style for beads.

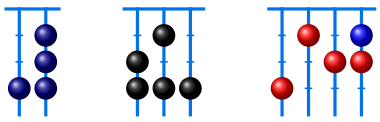


3B.2
```
\Abacus{3}{5,4,*1^2,0^4}
\Abacus[abacus star style={ball color=yellow}]
  {3}{5,4,*1^2,0^4}
```

runner bead height bead width

bead (default: ■) [accepts:]

The bead key sets the bead colour.



3B.3
```
\Abacus{2}{2^3,1}
\Abacus[bead=black]{3}{4^3,2,1}
\Abacus[bead=red]{4}{4^3,[ball color=blue]2,1}
```

The bead key changes the colour of every bead on the abacus. As shown, use the TikZ ball colour styling to change the colour of individual beads.

bead font (default:) [accepts:]

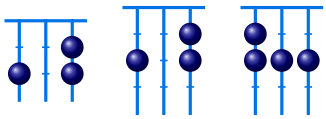
bead size (default: 0.3) [accepts: bead diameter in cm]

bead sep (default: 0.35) [accepts:]

bead text (default: white) [accepts:]

beta numbers (default: false) [accepts: false/true]

When beta numbers is true, the numbers in the bead specifications should be entered as *beta numbers*. The beta numbers give the bead positions, which are numbered $0, 1, \ldots$, starting from the first row and then continuing in this way in subsequent rows. Up to shift, bead numbers are first column hook lengths. In particular, beta numbers are pairwise distinct.



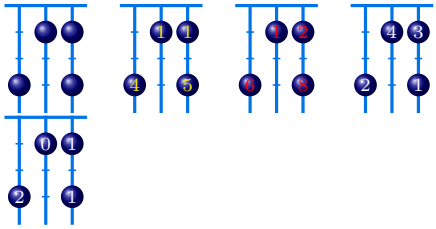
3B.4
```
\Abacus{3}{3,2^2}
\Abacus[beta numbers]{3}{5,3,2}
\Abacus[beta numbers]{3}{5,4,3,0}
```

runner (default:) [accepts:]

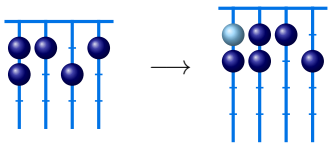
runner sep (default: 0.35) [accepts:]

tick (default:) [accepts:]

tick width (default: 0.1) [accepts:]



3B.5
```
\Abacus{3}{5,4,1,1}
\Abacus[entries=shape,bead text=yellow]{3}{5,4,1,1}
\Abacus[entries=beta,bead text=red]{3}{5,4,1,1}
\Abacus[entries=rows]{3}{5,4,1,1}
\Abacus[entries=residue]{3}{5,4,1,1}
```



3B.6
```
\tikzset{B/.style={ball color=LightSkyBlue}}
\Abacus[beta numbers]{4}{6,4,3,1,0}
      $\longrightarrow$
\Abacus[beta numbers]{4}{7,5,4,2,1,[B]0}
```

4. List of aTableau commands and options

The list of commands provided by the aTableau package is:

Command	Diagram	Section
\Abacus	An abacus for a partition	Chapter 3
\Diagram	A Young diagram for a partition	Section 2B
\Multidiagram	An ℓ-tuple of Young diagrams	Section 2G
\Multitableau	An ℓ-tuple of tableaux	Section 2G
\RibbonTableau	A ribbon tableau	Section 2F
\ShiftedDiagram	A shifted Young diagram	Section 2D
\ShiftedTableau	A shifted tableau	Section 2D
\SkewDiagram	A skew Young diagram	Section 2C
\SkewTableau	A skew tableau	Section 2C
\Tableau	A tableau	Section 2A
\Tabloid	A tabloid	Section 2E

In addition, the `\aTabset` command can be used to set default aTableau options.

For easier reference, here is the list of aTableau keys, together with a quick explanation of what they do. The last two columns indicate whether the options apply to tableau (and diagrams), or to abacuses — or both! Although not listed, American spelling variations of all keys are tolerated.

Key	Meaning	Tableau	Abacus
abacus star style	Set TikZ-style of abacus $*$-nodes	✗	✓
align	Set baseline alignment of aTableau diagram	✓	✓
australian	Use the Australian convention for tableau	✓	✗
bead height	Set row height between beads	✗	✓
bead width	Set column height between beads	✗	✓
bead		✗	✓
bead font		✗	✓
bead sep		✗	✓
bead size		✗	✓
bead text		✗	✓
beta numbers		✗	✓
border colour	Set tableau border colour	✓	✗
border style	Set tableau border style	✓	✗
border	Draw the tableau border	✓	✗
box fill	Set tableau box fill colour	✓	✗
box font	Set tableau box font	✓	✗
box height	Set tableau box height	✓	✗
box text	Set tableau box text colour	✓	✗
box width	Set tableau box width	✓	✗
boxes	Draw inner walls around boxes	✓	✗
cartan	Set Cartan type for residues	✓	✗
charge	Set charge for contents and residues	✓	✓
conjugate	Draw the conjugate tableau/diagram	✓	✗
delimiters	Set delimiters for multitableau and multidiagrams	✓	✗
dotted cols	Specify diagram columns to replace with dots	✓	✗
dotted columns	Specify diagram columns to replace with dots	✓	✗
dotted rows	Specify diagram rows to replace with dots	✓	✗
east	Abacus grows in the easterly direction	✓	✗
empty	Symbol for empty tableau in multitableau	✓	✗
english	Use the English convention for tableau	✓	✗
entries	Draw contents/residues/hooks/first/last tableau	✓	✗
e	Set the quantum characteristic for residues	✓	✓
french	Use the French convention for tableau	✓	✗
halign	Horizontal alignment of box entries and bead labels	✓	✓
inner colour	Set colour of inner tableau wall	✓	✗
inner style	Set style of inner tableau wall	✓	✗
label style	Set style of tableau labels	✓	✗
label	Set tableau label	✓	✗
left delimiter	Set left delimiter for multitableau	✓	✗
math boxes	Typeset box entries in math-mode (true by default)	✓	✗
name	Prefix of named nodes	✓	✓
no border	Disable tableau border wall	✓	✗
no boxes	Do not draw inner walls around boxes	✓	✗
no separators	Disable separators in multitableau	✓	✗
no skew border	Disable skew border	✓	✗
no skew boxes	Disable drawing of skew boxes	✓	✗
north	Abacus grows in the northerly direction	✓	✗
ribbon box	Set style of boxes in ribbons	✓	✗

Key	Meaning	Tableau	Abacus
ribbon style	Set ribbon style	✔	✗
ribbons	Comma separated list of ribbons to add to tableau	✔	✗
right delimiter	Set left delimiter for multitableau	✔	✗
rows	Set number of rows in multitableau or abacus	✔	✔
runner	Set the colour of the abacus runners	✗	✔
runner sep	Set distance between abacus runners	✗	✔
scale	Sets the aTableau scale	✔	✔
separation	Set distance between component tableau in multitableau	✔	✗
separator colour	Set separator colour in multitable	✔	✗
separators	Enable separators in multitableau	✔	✗
separator	Set separator colour in multitableau	✔	✗
shifted	Set true for a shifted tableau	✔	✗
skew border style	Set TikZ-style of the skew border	✔	✗
skew border	Set skew border colour	✔	✗
skew box style	Set TikZ-style of the skew boxes	✔	✗
skew boxes	Enable drawing of skew boxes	✔	✗
skew	Set inner skew shape	✔	✗
snobs	Comma separated list of snobs to add to tableau	✔	✗
south	Abacus grows in the southerly direction	✔	✗
star style	Set TikZ-style of tableau $*$-nodes	✔	✗
tabloid	Set true for a shifted tableau	✔	✗
text boxes	labels typeset in text-mode (false by default)	✔	✔
tick	Set the colour of the abacus runner ticks	✗	✔
tick width	Set the width of the abacus runners	✗	✔
tikz after	TikZ-code injected after abacus	✔	✔
tikz before	TikZ-code injected before abacus	✔	✔
tikzpicture	Add tikzpicture environment keys	✔	✗
ukrainian	Use the Ukrainian convention for tableau	✔	✗
valign	Set vertical alignment of box entries and bead labels	✔	✔
west	Abacus grows in the westerly direction	✔	✗
xoffsets	Set the x-offsets for component diagrams in a multitableau	✔	✗
xscale	Set the aTableau x-scale	✔	✔
yoffsets	Set the y-offsets for component diagrams in a multitableau	✔	✗
yscale	Set the aTableau y-scale	✔	✔

4A. **Colours.** The aTableau package defines and uses the following colours:

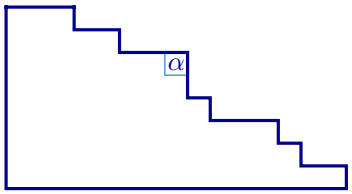
Colour	Name	HTML
	aTableauInner	0073E6
	aTableauMain	00008B
	aTableauSkewFill	F8F8F8
	aTableauSkew	818589
	aTableauStarStyle	E6F7FF

All of these colours are defined as HTML colours using the `\definecolor` command provided by `xcolor`.

It is possible to change the colours used in the diagrams created by aTableau by changing these colours. However, we recommend using the key-value interface instead.

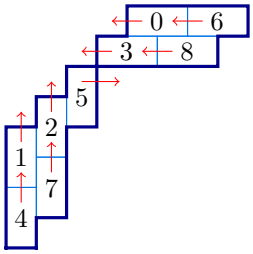
4A.1. *Examples from the literature.* We close this section showing how to draw some tableaux that appear in the literature. All of these diagrams predate this package. The aim of this section is not to showcase uses of this package but, rather, to show how some preexisting diagrams could have been drawn.

Diagrams like the following one are common. Note that the α is added in row 6 and column 8 using a ribbon.



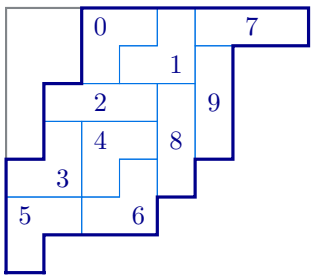
4A.1
```
\Diagram[french, no boxes,
  ribbons={*68_\alpha}, scale=0.6]
  {15,13,12,9,8^2,5,3}
```

I do not remember where I found the first example:

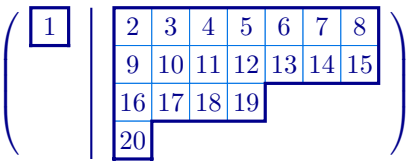


4A.2
```
\RibbonTableau[,skew={4,3,2,1},scale=0.8,
    tikz after = {
      % node labels
      \foreach \r/\c/\l in {1/6/0, 1/8/6, 2/5/3, 2/7/8}
        { \node at (B-\r-\c.west){\l};}
      \foreach \r/\c/\l in {5/1/1, 4/2/2, 3/3/5, 7/1/4, 6/2/7}
        { \node at (B-\r-\c.south){\l};}
      % arrows
      \foreach \a/\b in {1/5, 1/7, 2/4, 2/6}
        { \draw[red,->](B-\a-\b.center)--++(-0.4,0); }
      \draw[red,->](B-3-3.center)--++(0.5,0);
      \foreach \a/\b in {5/1, 7/1, 4/2, 6/2}
        { \draw[red,->](B-\a-\b.center)--++(0,0.4); }
    }
]{
    16c,18c, 25c, 27c, 33r, 42r, 51r, 62r,71r
}
```

These two ribbon tableaux can be found in Leclerc and Thibon's paper [**?**]

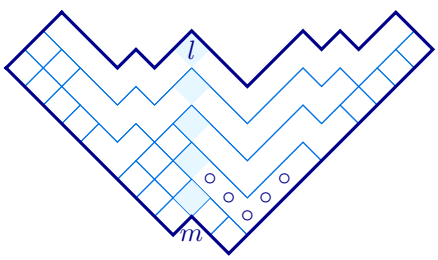


4A.3
```
\RibbonTableau[skew={2^2,1^2},skew border]{
   14c_0r,15r_1c, 18c_7c,26r_9r,
   34c_2c,35r_8r,42r_3c, 44c_4r,
   54r_6c,62c_5r
}
```



4A.4
```
\Multidiagram[entries=first,
  ribbon style={draw=red,thick},
  ribbons={24_{15}c_{14}, 26_{17}c_{16}, 27_{18},
    31_{11},  33_{13}c_{12}}]{1|7^2,4,1}
```

The following tableau comes from Fayers paper [**?**], which is uses Dyck tilings to understand homogeneous Garnir relations for KLR algenras of type A.



4A.5
```
\RibbonTableau[skew={1^2}, ukrainian, scale=0.7,
  ribbons={(draw=none)21_m}
]{
   12,16c_\circ c_\circ c_\circ r_\circ r_\circ *r,
   18crcccrr*r,19,1{10},1{11},1{12},
   22,29crcccrr*rccrcrr,
   2{12}ccrcrcccrr*r_lccrcrrr,
   31,*32,41,42,51,52,53,
   62cr,81,91,{10}1,{10}2,{11}1,{11}2
}
```

5. Feature requests and bug reports

Please make any feature requests and bug reports on the package github page

github.com/AndrewMathas/aTableau/.

Please give as much detail as possible when making such requests.

Bug reports must be a accompanied by a *minimal working example*, which is the smallest possible amount of LaTeX code that compiles and demonstrates the problem. This is necessary because if I cannot replicate your problem, then it is unlikely that I will be able to fix it.

Index